

# US Patent & Trademark Office

## Patent Public Search | Text View

---

|                      |                   |
|----------------------|-------------------|
| United States Patent | 12389046          |
| Kind Code            | B2                |
| Date of Patent       | August 12, 2025   |
| Inventor(s)          | Zhang; Kai et al. |

---

### Unsymmetric binary tree partitioning and non-dyadic blocks

---

#### Abstract

A mechanism for processing video data is disclosed. An Unsymmetric Binary Tree (UBT) partition of a parent block is determined to create two sub-blocks with different dimensions. At least one of the sub-blocks comprises a side that is non-dyadic. A conversion is performed between a visual media data and a bitstream based on the sub-blocks.

---

**Inventors:** Zhang; Kai (San Diego, CA), Zhang; Li (San Diego, CA), Deng; Zhipin (Beijing, CN), Zhang; Na (Beijing, CN), Wang; Yang (Beijing, CN)

**Applicant:** Beijing Bytedance Network Technology Co., Ltd. (Beijing, CN); Bytedance Inc. (Los Angeles, CA)

**Family ID:** 82932106

**Assignee:** BEIJING BYTEDANCE NETWORK TECHNOLOGY CO., LTD. (Beijing, CN); BYTEDANCE INC. (Los Angeles, CA)

**Appl. No.:** 18/451933

**Filed:** August 18, 2023

#### Prior Publication Data

| Document Identifier | Publication Date |
|---------------------|------------------|
| US 20230396812 A1   | Dec. 07, 2023    |

#### Foreign Application Priority Data

|    |                   |               |
|----|-------------------|---------------|
| WO | PCT/CN2021/077027 | Feb. 20, 2021 |
|----|-------------------|---------------|

#### Related U.S. Application Data

continuation parent-doc WO PCT/CN2022/076463 20220216 PENDING child-doc US 18451933

---

## Publication Classification

**Int. Cl.:** H04N19/96 (20140101); H04N19/119 (20140101); H04N19/176 (20140101);  
H04N19/70 (20140101)

**U.S. Cl.:**

**CPC** H04N19/96 (20141101); H04N19/119 (20141101); H04N19/176 (20141101);  
H04N19/70 (20141101);

## Field of Classification Search

**CPC:** H04N (19/96); H04N (19/119); H04N (19/176); H04N (19/70)

---

## References Cited

### U.S. PATENT DOCUMENTS

| Patent No.   | Issued Date | Patentee Name | U.S. Cl. | CPC        |
|--------------|-------------|---------------|----------|------------|
| 2019/0306538 | 12/2018     | Karczewicz    | N/A      | H04N 19/70 |
| 2020/0014946 | 12/2019     | Xu            | N/A      | H04N 19/44 |

### FOREIGN PATENT DOCUMENTS

| Patent No. | Application Date | Country | CPC |
|------------|------------------|---------|-----|
| 3446482    | 12/2018          | EP      | N/A |

### OTHER PUBLICATIONS

Document: JVET-K0287-v1, Gao, H., et al., "CE1-2.0.11: Picture Boundary Handling," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 11th Meeting: Ljubljana, SI, Jul. 10-18, 2018, 7 pages. cited by applicant

Sullivan, G., et al., "Overview of the High Efficiency Video Coding (HEVC) Standard," IEEE Transactions on Circuits and Systems for Video Technology, vol. 22, No. 12, Dec. 2012, 20 pages. cited by applicant

Luthra, A., et al., "Overview of the H.264/AVC video coding standard," Proceedings of SPIE vol. 5203 Applications of Digital Image Processing XXVI, Optical Science and Technology, SPIE's 48th Annual Meeting, 2003, San Diego, California, United States, Nov. 19, 2003, 16 pages. cited by applicant

Document: JVET-G1001-v1, Chen, J., et al., "Algorithm description of Joint Exploration Test Model 7 (JEM7)," Joint Video Exploration Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 7th Meeting: Torino, IT, Jul. 13-21, 2017, 50 pages. cited by applicant

JEM-7.0: Retrieved from the Internet:

[https://jvet.hhi.fraunhofer.de/svn/svn\\_HMJEMSoftware/tags/HM-16.6-JEM-7.0](https://jvet.hhi.fraunhofer.de/svn/svn_HMJEMSoftware/tags/HM-16.6-JEM-7.0), Nov. 15, 2023, 1 page. cited by applicant

"Series H: Audiovisual and Multimedia Systems Infrastructure of audiovisual services—Coding of moving video High efficiency video coding," Recommendation ITU-T H.265, Feb. 2018, 692 pages. cited by applicant

Document: JVET-D0117r1, Li, X., et al., "Multi-Type-Tree," Joint Video Exploration Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 4th Meeting: Chengdu, CN, Oct. 15-21, 2016, 3 pages. cited by applicant

Document: JVET-J1001-v2, Bross, B., "Versatile Video Coding (Draft 1)," Joint Video Experts

Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 10th Meeting: San Diego, US, Apr. 10-20, 2018, 43 pages. cited by applicant  
Document: JVET-J0022r1, Brodes, P., et al., “Description of SDR, HDR and 360°video coding technology proposal by Qualcomm and Technicolor—medium complexity version,” Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 10th Meeting: San Diego, US, Apr. 10-20, 2018, 84 pages. cited by applicant  
Document: JVET-K0197-v2, Leannec, F., et al., “CE1: Asymmetric Binary Tree (tests 1.0.1, 1.0.2, 1.0.3, 1.0.4, 8.0.1, 8.0.2),” Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 11th Meeting: Ljubljana, SI, Jul. 10-18, 2018, 20 pages. cited by applicant  
Document: JVET-D0064, Leannec, F., “Asymmetric Coding Units in QTBT,” Joint Video Exploration Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 4th Meeting: Chengdu, CN, Oct. 15-21, 2016, 10 pages. cited by applicant  
International Search Report from PCT application No. PCT/CN2022/076463 dated May 20, 2022, 10 pages. cited by applicant

---

*Primary Examiner:* Pham; Nam D

*Attorney, Agent or Firm:* Conley Rose, P.C.

---

## **Background/Summary**

CROSS-REFERENCE TO RELATED APPLICATIONS (1) This patent application is a continuation of International Application No. PCT/CN2022/076463 filed on Feb. 16, 2022, which claims the benefit of International Application No. PCT/CN2021/077027 filed Feb. 20, 2021. All the aforementioned patent applications are hereby incorporated by reference in their entireties.

### **TECHNICAL FIELD**

(1) This patent document relates to generation, storage, and consumption of digital audio video media information in a file format.

### **BACKGROUND**

(2) Digital video accounts for the largest bandwidth used on the Internet and other digital communication networks. As the number of connected user devices capable of receiving and displaying video increases, the bandwidth demand for digital video usage is likely to continue to grow.

### **SUMMARY**

(3) A first aspect relates to a method for processing video data comprising: determining an Unsymmetric Binary Tree (UBT) partition of a parent block to create two sub-blocks with different dimensions, wherein at least one of the sub-blocks comprises a side that is non-dyadic; and performing a conversion between a visual media data and a bitstream based on the sub-blocks.

(4) Optionally, in any of the preceding aspects, another implementation of the aspect provides that one of the sub-blocks is non-dyadic and one of the sub-blocks is dyadic.

(5) Optionally, in any of the preceding aspects, another implementation of the aspect provides that the UBT partition is a vertical UBT (UBT-V) partition, wherein the UBT-V partition is a UBT-V type 0 partition when a first sub-block is smaller than a second sub-block, and wherein the UBT-V partition is a UBT-V type 1 partition when the first sub-block is larger than the second sub-block.

(6) Optionally, in any of the preceding aspects, another implementation of the aspect provides that the UBT partition is a horizontal UBT (UBT-H) partition, wherein the UBT-H partition is a UBT-H type 0 partition when a first sub-block is smaller than a second sub-block, and wherein the UBT-H partition is a UBT-H type 1 partition when the first sub-block is larger than the second sub-block.

- (7) Optionally, in any of the preceding aspects, another implementation of the aspect provides determining to partition a non-dyadic block into sub-blocks.
- (8) Optionally, in any of the preceding aspects, another implementation of the aspect provides that a width (W) of the non-dyadic block is a non-dyadic number and a height (H) of the non-dyadic block is a dyadic number.
- (9) Optionally, in any of the preceding aspects, another implementation of the aspect provides that the partition of the non-dyadic block is a horizontal partition.
- (10) Optionally, in any of the preceding aspects, another implementation of the aspect provides that the partition of the non-dyadic block is a vertical partition.
- (11) Optionally, in any of the preceding aspects, another implementation of the aspect provides that the partition of the non-dyadic block results in a proximate sub-block with a width (W1) and a distal sub-block with a width (W2), and wherein W1 and W2 are dyadic, non-dyadic, or combinations thereof.
- (12) Optionally, in any of the preceding aspects, another implementation of the aspect provides that the partition of the non-dyadic block results in a proximate sub-block with a width (W1) and a distal sub-block with a width (W2), and wherein  $W1=W2=W/2$ ,  $W1=\sup.1<< \lfloor \log.\sup.2 .\sup.W \rfloor$  and  $W2=W-W1$ ,  $W2=\sup.1<< \lfloor \log.\sup.2 .\sup.W \rfloor$  and  $W1=W-W2$ ,  $W1=\sup.1<< \lceil \log.\sup.2 .\sup.W \rceil$  and  $W2=W-W1$ ,  $W2=\sup.1<< \lceil \log.\sup.2 .\sup.W \rceil$  and  $W1=W-W2$ ,  $W1=1<<N$  and  $W2=W-W1$  and  $N<\sup.1<< \lfloor \log.\sup.2 .\sup.W \rfloor$  where N is a number,  $W2=1<<N$  and  $W1=W-W2$  and  $N<\sup.1<< \lfloor \log.\sup.2 .\sup.W \rfloor$  where N is a number, or combinations thereof.
- (13) Optionally, in any of the preceding aspects, another implementation of the aspect provides that a width (W) of the non-dyadic block is a dyadic number and a height (H) of the non-dyadic block is a non-dyadic number.
- (14) Optionally, in any of the preceding aspects, another implementation of the aspect provides that the partition of the non-dyadic block is a vertical partition.
- (15) Optionally, in any of the preceding aspects, another implementation of the aspect provides that the partition of the non-dyadic block is a horizontal partition.
- (16) Optionally, in any of the preceding aspects, another implementation of the aspect provides that the partition of the non-dyadic block results in a proximate sub-block with a height (H1) and a distal sub-block with a height (H2), and wherein H1 and H2 are dyadic, non-dyadic, or combinations thereof.
- (17) Optionally, in any of the preceding aspects, another implementation of the aspect provides that the partition of the non-dyadic block results in a proximate sub-block with a height (H1) and a distal sub-block with a height (H2), and wherein  $H1=H2=H/2$ ,  $H1=\sup.1<< \lfloor \log.\sup.2 .\sup.H \rfloor$  and  $H2=H-H1$ ,  $H2=\sup.1<< \lfloor \log.\sup.2 .\sup.H \rfloor$  and  $H1=H-H2$ ,  $H1=\sup.1<< \lceil \log.\sup.2 .\sup.H \rceil$  and  $H2=H-H1$ ,  $H2=\sup.1<< \lceil \log.\sup.2 .\sup.H \rceil$  and  $H1=H-H2$ ,  $H1=1<<N$  and  $H2=H-H1$  and  $N<\sup.1<< \lfloor \log.\sup.2 .\sup.H \rfloor$  where N is a number,  $H2=1<<N$  and  $H1=H-H2$  and  $N<\sup.1<< \lfloor \log.\sup.2 .\sup.H \rfloor$  where N is a number, or combinations thereof.
- (18) Optionally, in any of the preceding aspects, another implementation of the aspect provides that a width (W) of the non-dyadic block is a non-dyadic number and a height (H) of the non-dyadic block is a non-dyadic number.
- (19) Optionally, in any of the preceding aspects, another implementation of the aspect provides that the partition of the non-dyadic block is a vertical partition.
- (20) Optionally, in any of the preceding aspects, another implementation of the aspect provides that the partition of the non-dyadic block results in a proximate sub-block with a width (W1) and a distal sub-block with a width (W2), and wherein W1 and W2 are dyadic, non-dyadic, or combinations thereof.
- (21) Optionally, in any of the preceding aspects, another implementation of the aspect provides that the partition of the non-dyadic block results in a proximate sub-block with a width (W1) and a distal sub-block with a width (W2), and wherein  $W1=W2=W/2$ ,  $W1=\sup.1<< \lfloor \log.\sup.2 .\sup.W \rfloor$

- and  $W2=W-W1$ ,  $W2=\sup.1<<\lfloor \log.\sup.2.\sup.W \rfloor$  and  $W1=W-W2$ ,  $W1=\sup.1<<\lceil \log.\sup.2.\sup.W \rceil$  and  $W2=W-W1$ ,  $W2=\sup.1<<\lceil \log.\sup.2.\sup.W \rceil$  and  $W1=W-W2$ ,  $W1=1<<N$  and  $W2=W-W1$  and  $N<\sup.1<<\lfloor \log.\sup.2.\sup.W \rfloor$  where  $N$  is a number,  $W2=1<<N$  and  $W1=W-W2$  and  $N<\sup.1<<\lfloor \log.\sup.2.\sup.W \rfloor$  where  $N$  is a number, or combinations thereof.
- (22) Optionally, in any of the preceding aspects, another implementation of the aspect provides that the partition of the non-dyadic block is a horizontal partition.
- (23) Optionally, in any of the preceding aspects, another implementation of the aspect provides that the partition of the non-dyadic block results in a proximate sub-block with a height ( $H1$ ) and a distal sub-block with a height ( $H2$ ), and wherein  $H1$  and  $H2$  are dyadic, non-dyadic, or combinations thereof.
- (24) Optionally, in any of the preceding aspects, another implementation of the aspect provides that the partition of the non-dyadic block results in a proximate sub-block with a height ( $H1$ ) and a distal sub-block with a height ( $H2$ ), and wherein  $H1=H2=H/2$ ,  $H1=\sup.1<<\lfloor \log.\sup.2.\sup.H \rfloor$  and  $H2=H-H1$ ,  $H2=\sup.1<<\lfloor \log.\sup.2.\sup.H \rfloor$  and  $H1=H-H2$ ,  $H1=\sup.1<<\lceil \log.\sup.2.\sup.H \rceil$  and  $H2=H-H1$ ,  $H2=\sup.1<<\lceil \log.\sup.2.\sup.H \rceil$  and  $H1=H-H2$ ,  $H1=1<<N$  and  $H2=H-H1$  and  $N<\sup.1<<\lfloor \log.\sup.2.\sup.H \rfloor$  where  $N$  is a number,  $H2=1<<N$  and  $H1=H-H2$  and  $N<\sup.1<<\lfloor \log.\sup.2.\sup.H \rfloor$  where  $N$  is a number, or combinations thereof.
- (25) Optionally, in any of the preceding aspects, another implementation of the aspect provides that a rule prevents partitioning a sub-block resulting from UBT.
- (26) Optionally, in any of the preceding aspects, another implementation of the aspect provides that different block split constraints are applied to non-dyadic blocks and dyadic blocks.
- (27) Optionally, in any of the preceding aspects, another implementation of the aspect provides that partitioning the non-dyadic block adds a bit depth greater than one when the non-dyadic block has a side with a measurement that is less than half a measurement of a corresponding side of a parent block, and wherein partitioning a non-dyadic block adds a bit depth of less than or equal to one when the non-dyadic sub-block has a side with a measurement that is greater than half a measurement of a corresponding side of a parent block.
- (28) Optionally, in any of the preceding aspects, another implementation of the aspect provides that a rule prevents vertical partitioning of any sub-block resulting from UBT-V.
- (29) Optionally, in any of the preceding aspects, another implementation of the aspect provides that a rule prevents vertical partitioning of any sub-block resulting from UBT-H.
- (30) Optionally, in any of the preceding aspects, another implementation of the aspect provides that a rule prevents horizontal partitioning of any sub-block resulting from UBT-V.
- (31) Optionally, in any of the preceding aspects, another implementation of the aspect provides that a rule prevents horizontal partitioning of any sub-block resulting from UBT-H.
- (32) Optionally, in any of the preceding aspects, another implementation of the aspect provides that a rule prevents all partitioning of any sub-block resulting from UBT.
- (33) Optionally, in any of the preceding aspects, another implementation of the aspect provides that a rule requires further partitioning of any sub-block resulting from UBT.
- (34) Optionally, in any of the preceding aspects, another implementation of the aspect provides that the bitstream comprises a syntax element that signals UBT, UBT-V, UBT-H, UBT type 0, UBT type 1, UBT split ratio, or combinations thereof.
- (35) A second aspect relates to a non-transitory computer readable medium comprising a computer program product for use by a video coding device, the computer program product comprising computer executable instructions stored on the non-transitory computer readable medium such that when executed by a processor cause the video coding device to perform the method of any of the preceding aspects.
- (36) A third aspect relates to an apparatus for processing video data comprising: a processor; and a non-transitory memory with instructions thereon, wherein the instructions upon execution by the processor, cause the processor to perform the method of any of the preceding aspects.

(37) For the purpose of clarity, any one of the foregoing embodiments may be combined with any one or more of the other foregoing embodiments to create a new embodiment within the scope of the present disclosure.

(38) These and other features will be more clearly understood from the following detailed description taken in conjunction with the accompanying drawings and claims.

---

## Description

### BRIEF DESCRIPTION OF THE DRAWINGS

(1) For a more complete understanding of this disclosure, reference is now made to the following brief description, taken in connection with the accompanying drawings and detailed description, wherein like reference numerals represent like parts.

(2) FIG. 1 is a schematic diagram of an example coding and decoding (codec) for video coding.

(3) FIG. 2 is a schematic diagram of example macroblock partitions.

(4) FIG. 3 is a schematic diagram of example modes for partitioning coding blocks, for example according to High Efficiency Video Coding (HEVC).

(5) FIG. 4 is a schematic diagram of example method for partitioning a picture for coding residual.

(6) FIG. 5 is a schematic diagram of example method for partitioning a picture, for example according to a quad tree binary tree (QTBT) structure.

(7) FIG. 6 is a schematic diagram of example partitioning structures used in Versatile Video Coding (VVC).

(8) FIG. 7 is a schematic diagram of example extended quad tree (EQT) partitioning structures.

(9) FIG. 8 is a schematic diagram of example flexible tree (FT) partitioning structures.

(10) FIG. 9 is a schematic diagram of example generalized TT (GTT) partitioning structures.

(11) FIG. 10 is a schematic diagram of example boundary partitioning tree.

(12) FIG. 11 is a schematic diagram of example partitioning structures used in Audio and Video Coding Standard (AVS) part three (AVS-3.0).

(13) FIG. 12 is a schematic diagram of example Unsymmetrical Quad-Tree (UQT) partitioning structures.

(14) FIG. 13 is a schematic diagram of another example of UQT partitioning structures.

(15) FIG. 14 is a schematic diagram of an example largest coding unit (LCU) partition with quad tree (QT), binary tree (BT), EQT, and UQT partitioning.

(16) FIG. 15 is a schematic diagram of an example tree type coding structure of an intra prediction (I) slice.

(17) FIG. 16 is a schematic diagram of an example tree type coding structure of a bidirectional inter prediction (B) and/or unidirectional inter prediction (P) slice.

(18) FIG. 17 is a schematic diagram of example Extended Ternary-Tree (ETT) partitioning structures.

(19) FIG. 18 is a schematic diagram of example  $\frac{1}{4}$  Unsymmetric Binary Tree (UBT) partitioning structures.

(20) FIG. 19 is a schematic diagram of example  $\frac{1}{8}$  UBT partitioning structures.

(21) FIG. 20 is a schematic diagram of example  $\frac{1}{16}$  UBT partitioning structures.

(22) FIGS. 21-22 are schematic diagrams of example non-dyadic partitions.

(23) FIG. 23 is a block diagram showing an example video processing system.

(24) FIG. 24 is a block diagram of an example video processing apparatus.

(25) FIG. 25 is a flowchart for an example method of video processing.

(26) FIG. 26 is a block diagram that illustrates an example video coding system.

(27) FIG. 27 is a block diagram that illustrates an example encoder.

(28) FIG. 28 is a block diagram that illustrates an example decoder.

(29) FIG. 29 is a schematic diagram of an example encoder.

#### DETAILED DESCRIPTION

(30) It should be understood at the outset that although an illustrative implementation of one or more embodiments are provided below, the disclosed systems and/or methods may be implemented using any number of techniques, whether currently known or yet to be developed. The disclosure should in no way be limited to the illustrative implementations, drawings, and techniques illustrated below, including the exemplary designs and implementations illustrated and described herein, but may be modified within the scope of the appended claims along with their full scope of equivalents.

(31) Versatile Video Coding (VVC), also known as H.266, terminology is used in some description only for ease of understanding and not for limiting scope of the disclosed techniques. As such, the techniques described herein are applicable to other video codec protocols and designs also. In the present document, editing changes are shown to text by bold italics indicating cancelled text and bold underline indicating added text, with respect to the VVC specification or International Organization for Standardization (ISO) base media file format (ISO/BMFF) file format specification.

(32) This document is related to image/video coding, and more particularly to the partition structure used in video coding. Partitioning includes mechanisms that split one large block to smaller blocks. The disclosed mechanisms may be applied to the video coding standards such as High Efficiency Video Coding (HEVC) and/or Versatile Video Coding (VVC). Such mechanisms may also be applicable to other video coding standards and/or video codecs.

(33) Video coding standards have evolved primarily through the development of the International Telecommunication Union (ITU) Telecommunication Standardization Sector (ITU-T) and International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC) standards. The ITU-T produced a H.261 standard and a H.263 standard, ISO/IEC produced Motion Picture Experts Group (MPEG) phase one (MPEG-1) and MPEG phase four (MPEG-4) Visual standards, and the two organizations jointly produced the H.262/MPEG phase two (MPEG-2) Video standard, the H.264/MPEG-4 Advanced Video Coding (AVC) standard, and the H.265/High Efficiency Video Coding (HEVC) standard. Since H.262, the video coding standards are based on a hybrid video coding structure that utilizes a temporal prediction plus a transform coding.

(34) FIG. 1 is a schematic diagram of an example coding and decoding (codec) for video coding, for example according to HEVC. For example, codec **100** provides functionality to support converting a video file into a bitstream by encoding and/or decoding pictures. Codec **100** is generalized to depict components employed in both an encoder and a decoder. Codec **100** receives a stream of pictures as a video signal **101** and partitions the pictures. Codec **100** then compresses the pictures in the video signal **101** into a coded bitstream when acting as an encoder. When acting as a decoder, codec system **100** generates an output video signal from the bitstream. The codec **100** includes a general coder control component **111**, a transform scaling and quantization component **113**, an intra-picture estimation component **115**, an intra-picture prediction component **117**, a motion compensation component **119**, a motion estimation component **121**, a scaling and inverse transform component **129**, a filter control analysis component **127**, an in-loop filters component **125**, a decoded picture buffer component **123**, and a header formatting and context adaptive binary arithmetic coding (CABAC) component **131**. Such components are coupled as shown. In FIG. 1, black lines indicate movement of data to be encoded/decoded while dashed lines indicate movement of control data that controls the operation of other components. The components of codec **100** may all be present in the encoder. The decoder may include a subset of the components of codec **100**. For example, the decoder may include the intra-picture prediction component **117**, the motion compensation component **119**, the scaling and inverse transform component **129**, the in-loop filters component **125**, and the decoded picture buffer component **123**. These components are

now described.

(35) The video signal **101** is a captured video sequence that has been partitioned into blocks of pixels by a coding tree. A coding tree employs various split modes to subdivide a block of pixels into smaller blocks of pixels. These blocks can then be further subdivided into smaller blocks. The blocks may be referred to as nodes on the coding tree. Larger parent nodes are split into smaller child nodes. The number of times a node is subdivided is referred to as the depth of the node/coding tree. The divided blocks can be included in coding units (CUs) in some cases. For example, a CU can be a sub-portion of a CTU that contains a luma block, red difference chroma (Cr) block(s), and a blue difference chroma (Cb) block(s) along with corresponding syntax instructions for the CU. The split modes may include a binary tree (BT), triple tree (TT), and a quad tree (QT) employed to partition a node into two, three, or four child nodes, respectively, of varying shapes depending on the split modes employed. The video signal **101** is forwarded to the general coder control component **111**, the transform scaling and quantization component **113**, the intra-picture estimation component **115**, the filter control analysis component **127**, and the motion estimation component **121** for compression.

(36) The general coder control component **111** is configured to make decisions related to coding of the images of the video sequence into the bitstream according to application constraints. For example, the general coder control component **111** manages optimization of bitrate/bitstream size versus reconstruction quality. Such decisions may be made based on storage space/bandwidth availability and image resolution requests. The general coder control component **111** also manages buffer utilization in light of transmission speed to mitigate buffer underrun and overrun issues. To manage these issues, the general coder control component **111** manages partitioning, prediction, and filtering by the other components. For example, the general coder control component **111** may increase compression complexity to increase resolution and increase bandwidth usage or decrease compression complexity to decrease resolution and bandwidth usage. Hence, the general coder control component **111** controls the other components of codec **100** to balance video signal reconstruction quality with bit rate concerns. The general coder control component **111** creates control data, which controls the operation of the other components. The control data is also forwarded to the header formatting and CABAC component **131** to be encoded in the bitstream to signal parameters for decoding at the decoder.

(37) The video signal **101** is also sent to the motion estimation component **121** and the motion compensation component **119** for inter-prediction. A video unit (e.g., a picture, a slice, a CTU, etc.) of the video signal **101** may be divided into multiple blocks. Motion estimation component **121** and the motion compensation component **119** perform inter-predictive coding of the received video block relative to one or more blocks in one or more reference pictures to provide temporal prediction. Codec system **100** may perform multiple coding passes, e.g., to select an appropriate coding mode for each block of video data.

(38) Motion estimation component **121** and motion compensation component **119** may be highly integrated, but are illustrated separately for conceptual purposes. Motion estimation, performed by motion estimation component **121**, is the process of generating motion vectors, which estimate motion for video blocks. A motion vector, for example, may indicate the displacement of a coded object in a current block relative to a reference block. A reference block is a block that is found to closely match the block to be coded, in terms of pixel difference. Such pixel differences may be determined by sum of absolute difference (SAD), sum of square difference (SSD), or other difference metrics. HEVC employs several coded objects including a CTU, coding tree blocks (CTBs), and CUs. For example, a CTU can be divided into CTBs, which can then be divided into CBs for inclusion in CUs. A CU can be encoded as a prediction unit (PU) containing prediction data and/or a transform unit (TU) containing transformed residual data for the CU. The motion estimation component **121** generates motion vectors, PUs, and TUs by using a rate-distortion analysis as part of a rate distortion optimization process. For example, the motion estimation



component **121** may determine multiple reference blocks, multiple motion vectors, etc. for a current block/frame, and may select the reference blocks, motion vectors, etc. having the best rate-distortion characteristics. The best rate-distortion characteristics balance both quality of video reconstruction (e.g., amount of data loss by compression) with coding efficiency (e.g., size of the final encoding).

(39) In some examples, codec **100** may calculate values for sub-integer pixel positions of reference pictures stored in decoded picture buffer component **123**. For example, video codec **100** may interpolate values of one-quarter pixel positions, one-eighth pixel positions, or other fractional pixel positions of the reference picture. Therefore, motion estimation component **121** may perform a motion search relative to the full pixel positions and fractional pixel positions and output a motion vector with fractional pixel precision. The motion estimation component **121** calculates a motion vector for a PU of a video block in an inter-coded slice by comparing the position of the PU to the position of a reference block of a reference picture. Motion estimation component **121** outputs the calculated motion vector as motion data to header formatting and CABAC component **131** for encoding and to the motion compensation component **119**.

(40) Motion compensation, performed by motion compensation component **119**, may involve fetching or generating a reference block based on the motion vector determined by motion estimation component **121**. Motion estimation component **121** and motion compensation component **119** may be functionally integrated, in some examples. Upon receiving the motion vector for the PU of the current video block, motion compensation component **119** may locate the reference block to which the motion vector points. A residual video block is then formed by subtracting pixel values of the reference block from the pixel values of the current block being coded, forming pixel difference values. In general, motion estimation component **121** performs motion estimation relative to luma components, and motion compensation component **119** uses motion vectors calculated based on the luma components for both chroma components and luma components. The reference block and residual block are forwarded to transform scaling and quantization component **113**.

(41) The video signal **101** is also sent to intra-picture estimation component **115** and intra-picture prediction component **117**. As with motion estimation component **121** and motion compensation component **119**, intra-picture estimation component **115** and intra-picture prediction component **117** may be highly integrated, but are illustrated separately for conceptual purposes. The intra-picture estimation component **115** and intra-picture prediction component **117** intra-predict a current block relative to blocks in a current picture, as an alternative to the inter-prediction performed by motion estimation component **121** and motion compensation component **119** between pictures, as described above. In particular, the intra-picture estimation component **115** determines an intra-prediction mode to use to encode a current block. In some examples, intra-picture estimation component **115** selects an appropriate intra-prediction mode to encode a current block from multiple tested intra-prediction modes. The selected intra-prediction modes are then forwarded to the header formatting and CABAC component **131** for encoding.

(42) For example, the intra-picture estimation component **115** calculates rate-distortion values using a rate-distortion analysis for the various tested intra-prediction modes, and selects the intra-prediction mode having the best rate-distortion characteristics among the tested modes. Rate-distortion analysis generally determines an amount of distortion (or error) between an encoded block and an original unencoded block that was encoded to produce the encoded block, as well as a bitrate (e.g., a number of bits) used to produce the encoded block. The intra-picture estimation component **115** calculates ratios from the distortions and rates for the various encoded blocks to determine which intra-prediction mode exhibits the best rate-distortion value for the block. In addition, intra-picture estimation component **115** may be configured to code depth blocks of a depth map using a depth modeling mode (DMM) based on rate-distortion optimization (RDO).

(43) The intra-picture prediction component **117** may generate a residual block from the reference

block based on the selected intra-prediction modes determined by intra-picture estimation component **115** when implemented on an encoder or read the residual block from the bitstream when implemented on a decoder. The residual block includes the difference in values between the reference block and the original block, represented as a matrix. The residual block is then forwarded to the transform scaling and quantization component **113**. The intra-picture estimation component **115** and the intra-picture prediction component **117** may operate on both luma and chroma components.

(44) The transform scaling and quantization component **113** is configured to further compress the residual block. The transform scaling and quantization component **113** applies a transform, such as a discrete cosine transform (DCT), a discrete sine transform (DST), or a conceptually similar transform, to the residual block, producing a video block comprising residual transform coefficient values. Wavelet transforms, integer transforms, sub-band transforms or other types of transforms could also be used. The transform may convert the residual information from a pixel value domain to a transform domain, such as a frequency domain. The transform scaling and quantization component **113** is also configured to scale the transformed residual information, for example based on frequency. Such scaling involves applying a scale factor to the residual information so that different frequency information is quantized at different granularities, which may affect final visual quality of the reconstructed video. The transform scaling and quantization component **113** is also configured to quantize the transform coefficients to further reduce bit rate. The quantization process may reduce the bit depth associated with some or all of the coefficients. The degree of quantization may be modified by adjusting a quantization parameter. In some examples, the transform scaling and quantization component **113** may then perform a scan of the matrix including the quantized transform coefficients. The quantized transform coefficients are forwarded to the header formatting and CABAC component **131** to be encoded in the bitstream.

(45) The scaling and inverse transform component **129** applies a reverse operation of the transform scaling and quantization component **113** to support motion estimation. The scaling and inverse transform component **129** applies inverse scaling, transformation, and/or quantization to reconstruct the residual block in the pixel domain, e.g., for later use as a reference block for another current block. The motion estimation component **121** and/or motion compensation component **119** may calculate a further reference block by adding the residual block back to a previous reference block for use in motion estimation of a later block/frame. Filters are applied to the reconstructed reference blocks to mitigate artifacts created during scaling, quantization, and transform. Such artifacts could otherwise cause inaccurate prediction (and create additional artifacts) when subsequent blocks are predicted.

(46) The filter control analysis component **127** and the in-loop filters component **125** apply the filters to the residual blocks and/or to reconstructed picture blocks. For example, the transformed residual block from the scaling and inverse transform component **129** may be combined with a corresponding reference block from intra-picture prediction component **117** and/or motion compensation component **119** to reconstruct the original image block. The filters may then be applied to the reconstructed image block. In some examples, the filters may instead be applied to the residual blocks. As with other components in FIG. 1, the filter control analysis component **127** and the in-loop filters component **125** are highly integrated and may be implemented together, but are depicted separately for conceptual purposes. Filters applied to the reconstructed reference blocks are applied to particular spatial regions and include multiple parameters to adjust how such filters are applied. The filter control analysis component **127** analyzes the reconstructed reference blocks to determine where such filters should be applied and sets corresponding parameters. Such data is forwarded to the header formatting and CABAC component **131** as filter control data for encoding. The in-loop filters component **125** applies such filters based on the filter control data. The filters may include a deblocking filter, a noise suppression filter, a SAO filter, and an adaptive loop filter. Such filters may be applied in the spatial/pixel domain (e.g., on a reconstructed pixel

block) or in the frequency domain, depending on the example.

(47) When operating as an encoder, the filtered reconstructed image block, residual block, and/or prediction block are stored in the decoded picture buffer component **123** for later use in motion estimation as discussed above. When operating as a decoder, the decoded picture buffer component **123** stores and forwards the reconstructed and filtered blocks toward a display as part of an output video signal. The decoded picture buffer component **123** may be any memory device capable of storing prediction blocks, residual blocks, and/or reconstructed image blocks.

(48) The header formatting and CABAC component **131** receives the data from the various components of codec **100** and encodes such data into a coded bitstream for transmission toward a decoder. Specifically, the header formatting and CABAC component **131** generates various headers to encode control data, such as general control data and filter control data. Further, prediction data, including intra-prediction and motion data, as well as residual data in the form of quantized transform coefficient data are all encoded in the bitstream. The final bitstream includes all information desired by the decoder to reconstruct the original partitioned video signal **101**. Such information may also include intra-prediction mode index tables (also referred to as codeword mapping tables), definitions of encoding contexts for various blocks, indications of most probable intra-prediction modes, an indication of partition information, etc. Such data may be encoded by employing entropy coding. For example, the information may be encoded by employing context adaptive variable length coding (CAVLC), CABAC, syntax-based context-adaptive binary arithmetic coding (SBAC), probability interval partitioning entropy (PIPE) coding, or another entropy coding technique. Following the entropy coding, the coded bitstream may be transmitted to another device (e.g., a video decoder) or archived for later transmission or retrieval.

(49) In order to encode and/or decode a picture as described above, the picture is first partitioned. FIG. 2 is a schematic diagram of example macroblock partitions **200**, which can be created by a partition tree structure pursuant to H.264/AVC. The core of the coding layer in such standards is the macroblock, containing a  $16 \times 16$  block of luma samples and, in the case of 4:2:0 color sampling, two corresponding  $8 \times 8$  blocks of chroma samples. An intra-coded block uses spatial prediction to exploit spatial correlation among pixels. Two partitions are defined for an intra-coded block, namely a  $16 \times 16$  sub-block and  $4 \times 4$  sub-block. An inter-coded block uses temporal prediction, instead of spatial prediction, by estimating motion among pictures. Motion can be estimated independently for either a  $16 \times 16$  macroblock or any sub-macroblock partitions. An inter-coded block can be partitioned into a  $16 \times 8$  sub-block, an  $8 \times 16$  sub-block, an  $8 \times 8$  sub-block, an  $8 \times 4$  sub-block, a  $4 \times 8$  sub-block, and/or a  $4 \times 4$  sub-block. All such values are measured in a number of samples. A Sample is a luma (light) value or chroma (color) value at a pixel.

(50) FIG. 3 is a schematic diagram of example modes **300** for partitioning coding blocks, for example according to HEVC. In HEVC, a picture is partitioned into CTUs. A CTU is split into CUs by using a quadtree structure denoted as a coding tree to adapt to various local characteristics. The decision whether to code a picture area using inter-picture (temporal) or intra-picture (spatial) prediction is made at the CU level. Each CU can be further split into one, two, or four PUs according to the PU splitting type. Inside one PU, the same prediction process is applied and the relevant information is transmitted to the decoder on a PU basis. After obtaining the residual block by applying the prediction process based on the PU splitting type, a CU can be partitioned into transform units (TUs) according to another quadtree structure similar to the coding tree for the CU. One feature of the HEVC structure is that HEVC has multiple partition conceptions including CU, PU, and TU.

(51) Various features involved in hybrid video coding using HEVC are highlighted as follows. HEVC includes the CTU, which is analogous to the macroblock in AVC. The CTU has a size selected by the encoder and can be larger than a macroblock. The CTU includes a luma coding tree block (CTB), corresponding chroma CTBs, and syntax elements. The size of a luma CTB, denoted as  $L \times L$ , can be chosen as  $L=16, 32$ , or  $64$  samples with the larger sizes resulting in better

compression. HEVC then supports a partitioning of the CTBs into smaller blocks using a tree structure and quadtree-like signaling.

(52) The quadtree syntax of the CTU specifies the size and positions of corresponding luma and chroma CBs. The root of the quadtree is associated with the CTU. Hence, the size of the luma CTB is the largest supported size for a luma CB. The splitting of a CTU into luma and chroma CBs is signaled jointly. One luma CB and two chroma CBs, together with associated syntax, form a coding unit (CU). A CTB may contain only one CU or may be split to form multiple CUs. Each CU has an associated partitioning into prediction units (PUs) and a tree of transform units (TUs). The decision of whether to code a picture area using inter picture or intra picture prediction is made at the CU level. A PU partitioning structure has a root at the CU level. Depending on the basic prediction-type decision, the luma and chroma CBs can then be further split in size and predicted from luma and chroma prediction blocks (PBs) according to modes **300**. HEVC supports variable PB sizes from  $64 \times 64$  down to  $4 \times 4$  samples. As shown, modes **300** can split a CB of size  $M$  pixels by  $M$  pixels into an  $M \times M$  block, a  $M/2 \times M$  block, a  $M \times M/2$  block, a  $M/2 \times M/2$  block, a  $M/4 \times M$  (left) block, a  $M/4 \times M$  (right) block, a  $M \times M/4$  (up) block, and/or a  $M \times M/4$  (down) block. It should be noted that the modes **300** for splitting CBs into PBs are subject to size constraints. Further, only  $M \times M$  and  $M/2 \times M/2$  are supported for intra picture predicted CBs.

(53) FIG. 4 is a schematic diagram of example method **400** for partitioning a picture for coding residual, for example according to HEVC. As noted above, blocks are coded by reference to reference blocks. A difference between values of a current block and the reference blocks is known as the residual. Method **400** is employed to compress the residual. For example, the prediction residual is coded using block transforms. Method **400** employs a TU tree structure **403** to partition a CTB **401** and included CBs for application of transform blocks (TBs). Method **400** illustrates the subdivision of a CTB **401** into CBs and TBs. Solid lines indicate CB boundaries and dotted lines indicate TB boundaries. The TU tree structure **403** is an example quadtree that partitions the CTB **401**. A transform, such as discrete cosine transform (DCT), is applied to each TB. The transform converts the residual into transform coefficients that can be represented using less data than the uncompressed residual. The TU tree structure **403** has a root at the CU level. The luma CB residual area may be identical to the luma TB area or may be further split into smaller luma TBs. The same applies to the chroma TBs. Integer basis transform functions similar to those of a DCT are defined for the square TB sizes  $4 \times 4$ ,  $8 \times 8$ ,  $16 \times 16$ , and  $32 \times 32$ . For the  $4 \times 4$  transform of luma intra picture prediction residuals, an integer transform derived from a form of DST is alternatively specified.

(54) A quadtree plus binary tree block structure with larger CTUs in Joint Exploration Model (JEM) is discussed below. Joint Video Exploration Team (JVET) was founded by Video Coding Experts group (VCEG) and MPEG to explore video coding technologies beyond HEVC. JVET has adopted many improvements included such improvements into a reference software named Joint Exploration Model (JEM).

(55) FIG. 5 is a schematic diagram of example method **500** for partitioning a picture, for example according to a quad tree binary tree (QTBT) structure **501**. A tree representation **503** of the QTBT structure **501** is also shown. Unlike the partitioning structures in HEVC, the QTBT structure **501** removes the concepts of multiple partition types. For example, the QTBT structure **501** removes the separation of the CU, PU, and TU concepts, and supports more flexibility for CU partition shapes. In the QTBT structure **501**, a CU can have either a square or rectangular shape. In method **500**, a CTU is first partitioned by a quadtree structure. The quadtree leaf nodes are further partitioned by a binary tree structure. Symmetric horizontal splitting and symmetric vertical splitting are two splitting types used in the binary tree. The binary tree leaf nodes are called CUs, and that segmentation is used for prediction and transform processing without further partitioning. This causes the CU, PU, and TU to have the same block size in the QTBT structure **501**. In the JEM, a CU sometimes includes CBs of different color components. For example, one CU may contain one luma CB and two chroma CBs in the case of unidirectional inter prediction (P) and

bidirectional inter prediction (B) slices of the 4:2:0 chroma format. Further, the CU sometimes includes a CB of a single component. For example, one CU may contain only one luma CB or just two chroma CBs in the case of intra prediction (I) slices.

(56) The following parameters are defined for the QTBT partitioning scheme. The CTU size is the root node size of a quadtree, which is the same concept as in HEVC. Minimum quad tree size (MinQTSIZE) is the minimum allowed quadtree leaf node size. Maximum binary tree size (MaxBTSIZE) is the maximum allowed binary tree root node size. Maximum binary tree depth (MaxBTDepth) is the maximum allowed binary tree depth. Minimum binary tree size (MinBTSIZE) is the minimum allowed binary tree leaf node size.

(57) In one example of the QTBT structure **501**, the CTU size is set as  $128 \times 128$  luma samples with two corresponding  $64 \times 64$  blocks of chroma samples, the MinQTSIZE is set as  $16 \times 16$ , the MaxBTSIZE is set as  $64 \times 64$ , the MinBTSIZE (for both width and height) is set as  $4 \times 4$ , and the MaxBTDepth is set as 4. The quadtree partitioning is applied to the CTU first to generate quadtree leaf nodes. The quadtree leaf nodes may have a size from  $16 \times 16$  (the MinQTSIZE) to  $128 \times 128$  (the CTU size). If the leaf quadtree node is  $128 \times 128$ , the node is not to be further split by the binary tree since the size exceeds the MaxBTSIZE (e.g.,  $64 \times 64$ ). Otherwise, the leaf quadtree node can be further partitioned by the binary tree. Therefore, the quadtree leaf node is also the root node for the binary tree and has the binary tree depth as 0. When the binary tree depth reaches MaxBTDepth (e.g., 4), no further splitting is considered. When the binary tree node has width equal to MinBTSIZE (e.g., 4), no further horizontal splitting is considered. Similarly, when the binary tree node has a height equal to MinBTSIZE, no further vertical splitting is considered. The leaf nodes of the binary tree are further processed by prediction and transform processing without any further partitioning. In the JEM, the maximum CTU size is  $256 \times 256$  luma samples.

(58) Method **500** illustrates an example of block partitioning by using the QTBT structure **501**, and tree representation **503** illustrates the corresponding tree representation. The solid lines indicate quadtree splitting and dotted lines indicate binary tree splitting. In each splitting (e.g., non-leaf) node of the binary tree, one flag is signalled to indicate which splitting type (e.g., horizontal or vertical) is used, where 0 indicates horizontal splitting and 1 indicates vertical splitting. For the quadtree splitting, there is no need to indicate the splitting type since quadtree splitting always splits a block both horizontally and vertically to produce 4 sub-blocks with an equal size.

(59) In addition, the QTBT scheme supports the ability for the luma and chroma to have a separate QTBT structure **501**. For example, in P and B slices the luma and chroma CTBs in one CTU share the same QTBT structure **501**. However, in I slices the luma CTB is partitioned into CUs by a QTBT structure **501**, and the chroma CTBs are partitioned into chroma CUs by another QTBT structure **501**. Accordingly, a CU in an I slice can include a coding block of the luma component or coding blocks of two chroma components. Further, a CU in a P or B slice includes coding blocks of all three color components. In HEVC, inter prediction for small blocks is restricted to reduce the memory access of motion compensation, such that bi-prediction is not supported for  $4 \times 8$  and  $8 \times 4$  blocks, and inter prediction is not supported for  $4 \times 4$  blocks. In the QTBT of the JEM, these restrictions are removed.

(60) Triple-tree partitioning for VVC is now discussed. FIG. **6** is a schematic diagram **600** of example partitioning structures used in VVC. As shown, split types other than quad-tree and binary-tree are supported in VVC. For example, schematic diagram **600** includes a quad tree partition **601**, a vertical binary tree partition **603**, a horizontal binary tree partition **605**, a vertical triple tree partition **607**, and a horizontal triple tree partition **609**. This approach introduces two triple tree (TT) partitions in addition to the quad tree and binary trees.

(61) In an example implementation, two levels of trees are employed including a region tree (a quad-tree) and a prediction tree (binary-tree or triple-tree). A CTU is first partitioned by a region tree (RT). A RT leaf may be further split with prediction tree (PT). A PT leaf may also be further split with PT until a max PT depth is reached. A PT leaf is a basic coding unit. The PT may also be

called a CU for convenience. In an example implementation, a CU cannot be further split. Prediction and transform are both applied on CU in the same way as JEM. The whole partition structure is named multiple-type-tree.

(62) An extended quad tree is now discussed. FIG. 7 is a schematic diagram **700** of example EQT partitioning structures. An EQT partitioning structure corresponding to a block partitioning process includes an extended quad tree partitioning process for the block of video data. The extended quad partitioning structure represents partitioning the block of video data into final sub-blocks. When the extended quad tree partitioning process decides to apply an extended quad tree partition to a block, the block is always split into four sub-blocks. Decoding of the final sub-blocks is based on the video bitstream. Decoding of the block of video data is based on the final sub-blocks decoded according to the EQT structure derived.

(63) The EQT partitioning process can be applied to a block recursively to generate EQT leaf nodes. Alternatively, when EQT is applied to a certain block, for each of the sub-blocks resulting from the EQT split, may further be split into BT and/or QT and/or TT and/or EQT and/or other kinds of partition trees. In one example, EQT and QT may share the same depth increment process and the same restrictions of leaf node sizes. In this case, the partitioning of one node can be implicitly terminated when the size of the node reaches a minimum allowed quad tree leaf node size or EQT depth with the node reaches a maximum allowed quad tree depth. Alternatively, EQT and QT may share different depth increment processes and/or restrictions of leaf node sizes. The partitioning of one node by EQT may be implicitly terminated when the size of the node reaches a minimum allowed EQT leaf node size or the EQT depth associated with the node reaches a maximum allowed EQT depth. In one example, the EQT depth and/or the minimum allowed EQT leaf node sizes may be signaled in a sequences parameter set (SPS), a picture parameter set (PPS), a slice header, a CTU, a region, a tile, and/or a CU.

(64) EQT may not use a quad tree partition applied to a square block, for example where the block has a size of  $M \times N$  where  $M$  and  $N$  are equal or unequal non-zero positive integer values. Instead, EQT splits one block equally into four partitions, such as an  $M/4 \times N$  split **701** or an  $M \times N/4$  split **703**. Split **727** and split **729** show general examples of split **701** and **703**, respectively. For example, split **727** is split into  $M \times N_1$ ,  $M \times N_2$ ,  $M \times N_3$ , and  $M \times N_4$ , where  $N_1 + N_2 + N_3 + N_4 = N$ . Further, split **729** is split into  $M_1 \times N$ ,  $M_2 \times N$ ,  $M_3 \times N$  and  $M_4 \times N$  where  $M_1 + M_2 + M_3 + M_4 = M$ .

(65) In another example, the EQT can split the shape equally into four partitions where the partition size is dependent on the maximum and minimum values of  $M$  and  $N$ . In one example, one  $4 \times 32$  block may be split into four  $4 \times 8$  sub-blocks while a  $32 \times 4$  block may be split into four  $8 \times 4$  sub-blocks.

(66) In another example, EQT splits one block equally into four partitions, such as two partitions are with size equal to  $(M \times w_0/w) \times (N \times h_0/h)$  and the other two are with  $(M \times (w-w_0)/w) \times (N \times (h-h_0)/h)$  as shown by split **705**, split **707**, split **709**, and split **711**. For example,  $w_0$  and  $w$  may be equal to 1 and 2, respectively, such that the width is reduced by half while the height can use other ratios instead of 2:1 to get the sub-blocks. In another example,  $h_0$  and  $h$  may be equal to 1 and 2, respectively, such that the height is reduced by half while the width can use other ratios instead of 2:1. For example, split **705** includes a sub-block width fixed to be  $M/2$  with a height equal to  $N/4$  or  $3N/4$  with a smaller selection for the top two partitions. For example, split **707** includes a sub-block height fixed to be  $N/2$  with a width equal to  $M/4$  or  $3M/4$  with a smaller selection for the left two partitions. For example, split **709** includes a sub-block width fixed to be  $M/2$  with a height equal to  $N/4$  or  $3N/4$  with a smaller selection for the bottom two partitions. For example, split **711** includes a sub-block height fixed to be  $N/2$  with a width equal to  $M/4$  or  $3M/4$  with a smaller selection for the right two partitions.

(67) Split **713**, split **715**, split **717**, split **719**, split **721**, and split **723** show other examples of quad tree partitioning. For example, split **713**, split **715**, and split **717** show options where the shape is split by  $M \times N/4$  and  $M/2 \times N/2$ . For example, split **719**, split **721**, and split **723** show options where

the shape is split by  $N \times M/4$  and  $N/2 \times M/2$ .

(68) Split **725** shows a more general case of quad tree partitioning with different shapes of partitions. In this case, split **725** is split such that  $M1 \times N1$ ,  $(M-M1) \times N1$ ,  $M1 \times (N-N1)$  and  $(M-M1) \times (N-N1)$ .

(69) FIG. **8** is a schematic diagram **800** of example flexible tree (FT) partitioning structures. A FT partitioning structure corresponds to a block partitioning process including an FT partitioning process for the block of video data. The FT partitioning structure represents a partitioning for a block of video data into final sub-blocks. When the FT partitioning process decides to apply a FT partition to a block, the block is split into K sub-blocks wherein K could be larger than 4. The final sub-blocks can be coded based on the video bitstream. Further, the block of video data can be decoded based on the final sub-blocks decoded according to the FT structure derived. The FT partitioning process can be applied to a given block recursively to generate FT tree leaf nodes. The partitioning of one node is implicitly terminated when the node reaches a minimum allowed FT leaf node size or FT depth associated with the node reaches a maximum allowed FT depth. Further, when FT is applied to a certain block, multiple sub-blocks can be created. Each of the sub-blocks created by FT may further be split into BT, QT, EQT, TT, and/or other kinds of partition trees. In an example, the FT depth or the minimum allowed FT leaf node sizes or the minimum allowed partition size for FT may be signaled in a SPS, a PPS, a slice header, a CTU, a region, a tile, and/or a CU. Similar to EQT, all of the sub-blocks created by FT partitions may be the same or different sizes.

(70) Schematic diagram **800** includes example FT partitioning structures where the number of sub-blocks, denoted as K, is set equal to six or eight. Split **801** is a partitioning structure with  $K=8$ ,  $M/4 \times N/2$ . Split **803** is a partitioning structure with  $K=8$ ,  $M/2 \times N/4$ . Split **805** is a partitioning structure with  $K=6$ ,  $M/2 \times N/2$  and  $M/4 \times N/2$ . Split **807** is a partitioning structure with  $K=6$ ,  $M/2 \times N/2$  and  $M/2 \times N/4$ .

(71) FIG. **9** is a schematic diagram **900** of example generalized TT (GTT) partitioning structures. For the TT partitioning structure, the restriction of splitting along either horizontal or vertical may be removed. The GTT partition pattern may be defined as splitting for both horizontal and vertical. Split **901** employs a left split from a vertical TT split and a horizontal BT split of the remaining area. Split **903** employs a bottom split from a horizontal TT split and a vertical BT split of the remaining area. In some examples, the partitioning EQT, FT, and/or GTT partitioning methods may be applied under certain conditions. In other words, when the condition(s) are not satisfied, there is no need to signal the partition types. In another example, the EQT, FT, and/or GTT partitioning methods may be used to replace other partition tree types. In another example, the EQT, FT, and/or GTT partitioning methods may be only used as a replacement for other partition tree types under certain conditions. In one example, the condition may be based on the picture, slice types, block sizes, the coded modes; and/or whether a block is located at a picture, slice, and/or tile boundary. In one example, EQT may be treated in the same way as QT. In this case, when the QT partition tree type is selected, more flags/indications of the detailed quad-tree partition patterns may be further signaled. In some examples, EQT may be treated as additional partition patterns. In one example, the signaling of partitioning methods of EQT, FT, and/or GTT may be conditional. For example, one or more EQP, FT, and/or GTT partitioning methods may not be used in some cases, and the bits corresponding to signal these partitioning methods are not signaled.

(72) FIG. **10** is a schematic diagram of example boundary partitioning tree **1000**, which is also known as a versatile boundary partition. The boundary partitioning tree **1000** is an example boundary handling method for VVC and/or Audio and Video Coding Standard Workgroup Part three (AVS-3.0). Since the forced quadtree boundary partition solution in VVC is not optimized, the boundary partitioning tree **1000** uses regular block partition syntax to maintain continuity with the CABAC engine as well as to match the picture boundary. The versatile boundary partition obtains the following rules (both encoder and decoder). Since the boundary partitioning tree **1000**

uses exactly the same partition syntax of the normal block (non-boundary) for boundaries located at block, the syntax is not changed. If the no split mode is parsed for the boundary CU, the forced boundary partition (FBP) is used to match the picture boundary. After the forced boundary partition is used (non-singling boundary partition), no further partition is performed. The forced boundary partition is described as follows. If the size of block is larger than the maximal allowed BT size, forced QT is used to perform the FBP in the current forced partition level. Otherwise, if the bottom-right sample of current CU is located below the bottom picture boundary and not extended to the right boundary, a forced horizontal BT is used to perform the FBP in the current forced partition level. Otherwise, if the bottom-right sample of current CU is located at the right side of the right picture boundary and not below the bottom boundary, a forced vertical BT is used to perform the FBP in the current forced partition level. Otherwise, if the bottom-right sample of current CU is located at the right side of the right picture boundary and below the bottom boundary, a forced QT is used to perform the FBP in the current forced partition level.

(73) FIG. **11** is a schematic diagram **1100** of example partitioning structures used in Audio and Video Coding Standard (AVS) part three (AVS-3.0). Partitioning in AVS-3.0 is now discussed. The Audio and Video Coding Standard (AVS) Workgroup of China was authorized to be established by the Science and Technology Department under the former Ministry of Industry and Information Technology of People's Republic of China. With the mandate of satisfying the demands from the rapidly growing information industry, AVS is committed to producing technical standards of high quality for compression, decompression, processing, and representation of digital audio and video, and thus providing digital audio-video equipment and systems with high-efficient and economical coding/decoding technologies. AVS can be applied in wide variety of significant information sectors including high-resolution digital broadcast, high-density laser-digital storage media, wireless broad-band multimedia communication and internet broad-band stream media. AVS is one of the second generation of source coding/decoding standards and owns independent Chinese intellectual property rights. Source coding technology primarily addresses the problem of coding and compressing audio and video mass data from initial data and original sources. Hence AVS is known as digital video and audio coding technology, and is the premise of the subsequent digital transmission, storage, and broadcast. Further, AVS serves as a common standard for the digital video and audio industry.

(74) AVS-3.0 employs a QT partitioning **1101**, a vertical BT partitioning **1105**, a horizontal BT partitioning **1103**, and a horizontal extended quad-tree (EQT) partitioning **1107**, and a vertical EQT partitioning **1109** to split a largest coding unit (LCU) into multiple CUs. QT partitioning, BT partitioning, and EQT partitioning can all be used for the root, internal nodes, or leaf nodes of the partitioning tree. However, QT partitioning is forbidden after any BT and/or EQT partitioning.

(75) FIG. **12** is a schematic diagram **1200** of example Unsymmetrical Quad-Tree (UQT) partitioning structures. UQT partitioning employs a block with dimensions  $W \times H$ , which is split into four partitions with dimensions  $W_1 \times H_1$ ,  $W_2 \times H_2$ ,  $W_3 \times H_3$  and  $W_4 \times H_4$ , where  $W_1$ ,  $W_2$ ,  $W_3$ ,  $W_4$ ,  $H_1$ ,  $H_2$ ,  $H_3$ ,  $H_4$  are all integers. In one example, and at least one of the partitions has different block size compared to others. In one example, only two of the four partitions may have equal size, and the other two are different with each other and different from the two partitions with equal size. In one example, all the parameters are in the form of power of 2. For example,  $W_1=2N_1$ ,  $W_2=2N_2$ ,  $W_3=2N_3$ ,  $W_4=2N_4$ ,  $H_1=2M_1$ ,  $H_2=2M_2$ ,  $H_3=2M_3$ ,  $H_4=2M_4$ . In one example, UQT only splits one partition in vertical direction, for example,  $H_1=H_2=H_3=H_4=H$ . In one example, in split **1201**  $W_1=W/8$ ,  $W_2=W/2$ ,  $W_3=W/8$ ,  $W_4=W/4$ ,  $H_1=H_2=H_3=H_4=H$ . This kind of UQT is vertical split and named as UQT1-V. In one example, in split **1203**  $W_1=W/8$ ,  $W_2=W/2$ ,  $W_3=W/4$ ,  $W_4=W/8$ ,  $H_1=H_2=H_3=H_4=H$ . This kind of UQT is vertical split and named as UQT2-V. In one example in split **1205**  $W_1=W/4$ ,  $W_2=W/8$ ,  $W_3=W/2$ ,  $W_4=W/8$ ,  $H_1=H_2=H_3=H_4=H$ . This kind of UQT is vertical split and named as UQT3-V. In one example, in split **1207**  $W_1=W/8$ ,  $W_2=W/4$ ,  $W_3=W/2$ ,  $W_4=W/8$ ,  $H_1=H_2=H_3=H_4=H$ . This kind of UQT is vertical split and named as UQT4-V.



(76) In one example, UQT only splits one partition in horizontal direction, for example,  $W1=W2=W3=W4=W$ . In one example, in split **1209**  $H1=H/8$ ,  $H2=H/2$ ,  $H3=H/8$ ,  $H4=H/4$ ,  $W1=W2=W3=W4=W$ . This kind of UQT is horizontal split and named as UQT1-H. In one example, in split **1211**  $H1=H/8$ ,  $H2=H/2$ ,  $H3=H/4$ ,  $H4=H/8$ ,  $W1=W2=W3=W4=W$ . This kind of UQT is horizontal split and named as UQT2-H. In one example, in split **1213**  $H1=H/4$ ,  $H2=H/8$ ,  $H3=H/2$ ,  $H4=H/8$ ,  $W1=W2=W3=W4=W$ . This kind of UQT is horizontal split and named as UQT3-H. In one example, in split **1215**  $H1=H/8$ ,  $H2=H/4$ ,  $H3=H/2$ ,  $H4=H/8$ ,  $W1=W2=W3=W4=W$ . This kind of UQT is horizontal split and named as UQT4-H.

(77) A block which is split into child blocks by UQT, may be split from a parent block by QT, BT, TT, or UQT. A block which may allow UQT partitions, may be a block generated by QT, BT, TT, or UQT partitions. A block which may allow UQT partitions, may be a block generated by QT, EQT, or BT partitions. A block which may allow UQT partitions, may be a block generated by UQT, QT, EQT or BT partitions. For example, the maximum block that allows UQT partitions may be the largest coding block (e.g., coding tree block or coding tree unit). For example, the maximum block that allows UQT partitions may be the virtual pipeline data unit (VPDU). For example, a block which allows UQT partitions may be split from a parent block by one or some specific kinds of split methods. For example, a block which is split into child blocks by UQT can only be split from a parent block by QT. In one example, the maximum/minimum block size that could allow UQT partitions and/or the maximum bit depth that could allow UQT partitions may be signaled in a SPS, a PPS, a video parameter set (VPS), an adaptation parameter set (APS), a sequence header, a picture header, a slice header, a tile group header, a CTU row/regions, etc. In one example, the maximum/minimum block size that could allow UQT partitions and/or the maximum bit depth that could allow UQT partitions may be dependent of profile/level/tier of a standard. In one example, the maximum/minimum block size that could allow UQT partitions and/or the maximum bit depth that could allow UQT partitions may be derived, for example to be the same as the allowed QT partitions.

(78) A block which is split from a parent block by UQT may be further split into child blocks by QT, BT, TT, and/or UQT. For example, a block which is split from a parent block by UQT may be further split into child blocks by BT. For example, a block which is split from a parent block by UQT may be further split into child blocks by TT. For example, a block which is split from a parent block by UQT may be further split into child blocks by UQT and/or QT. For example, a block which is split from a parent block by UQT cannot be further split into child blocks by QT. For example, a block which is split from a parent block by UQT may be further split into child blocks by EQT. In an example, a block split from a parent block by UQT may be further split into child blocks by UQT, EQT, and BT. For example, a block which is split from a parent block by UQT is disallowed to be further split into child blocks by UQT. In an example, a block split from a parent block by UQT may be further split into child blocks by EQT and BT.

(79) When a parent block is split into child blocks by UQT, the split depth of the child block may be derived from the split depth of the parent block. In one example, the splitting due to UQT may be used to update the QT, BT, TT, UQT, and/or MTT depth. In one example, the QT depth of one or all of the child blocks is equal to the QT depth of the parent block plus one. In one example, the BT depth of one or all of the child blocks is equal to the BT depth of the parent block plus one. In one example, the TT depth of one or all of the child blocks is equal to the TT depth of the parent block plus one. In one example, the UQT depth of one or all of the child blocks is equal to the UQT depth of the parent block plus one. In one example, the MTT depth of one or all of the child block is equal to the MTT depth of the parent block plus one. For example, the MTT depth of the child block is equal to the MTT depth of the parent block plus one if the parent block is split into child blocks by BT. For example, the MTT depth of the child block is equal to the MTT depth of the parent block plus one if the parent block is split into child blocks by TT. In an example, the MTT depth of one or all of the child block is equal to the MTT depth of the parent block plus K where

$K > 1$ . For example,  $K = 2$ . In one example,  $K$  may be different for different child blocks. In one example,  $K = \log 2$  (the size of the parent block/the size of the child block). For example, in split **1203** Ks for the four child blocks with  $W1 = W/8$ ,  $W2 = W/2$ ,  $W3 = W/4$  and  $W4 = W/8$  are 3, 1, 2, 3, respectively. In one example, the UQT, BT, TT, QT, and/or MTT depth increase for different child block may be different. The depth increase is dependent on the ratio of a child block compared to the parent block.

(80) Whether and how to use UQT may depend on color format (such as 4:4:4 or 4:2:0) and/or color components. Whether and how to use UQT may depend on whether luma and chroma coding trees are separated. In one example, UQT can only be applied on the luma component when luma and chroma coding trees are separated. Whether to apply UQT and/or which kind UQT is applied may be signaled from encoder to decoder. In one example, the VPS, SPS, PPS, picture header, slice header, tile group header, and/or tile header can be used to signal whether UQT can be applied. In one example, the VPS, SPS, PPS, picture header, slice header, tile group header, and/or tile header can be used to signal which kinds of UQT can be applied. In one example, whether UQT is used to split a block may be signaled in the block. In one example, which kind of UQT is used to split a block may be signaled in the block. In one example, an index of partition type may be signaled in a block to indicate whether the block is split by QT, UQT, or a non-split. In an example, the splitting direction (horizontal/vertical) and/or splitting patterns may be further signaled. In one example, an index of a partition type may be signaled in a block to indicate whether a block is split by BT, TT, or UQT. For example, this index may be conditionally signaled, such as only when at least one of BT, TT, and UQT is valid for the block. In an example, the splitting direction (horizontal/vertical) and/or splitting patterns may be further signaled. In one example, indication of splitting direction may be signaled first and followed by the splitting pattern (such as QT, TT, UQT). In one example, a flag is signaled in a block to indicate whether a block is vertical split or horizontal split. The vertical split may be a BT vertical split, a TT vertical split, or UQT vertical split. The horizontal split may be BT horizontal split, TT horizontal split, or UQT horizontal split. For example, this flag is signaled only when the block is split by BT, TT, or UQT. For example, this flag is signaled only when both vertical split and horizontal split are valid for this block. If only vertical split is valid, the flag is not signaled and horizontal split is inferred to be used. If only horizontal split is valid, the flag is not signaled and vertical split is inferred to be used. In one example, a binarized code is signaled in a block to indicate which kind of split (BT, TT, or a kind of UQT) is used. In the following examples,  $X$  represents 0 or 1 and  $Y = \sim X$  ( $Y = 1$  if  $X = 0$  and  $Y = 0$  if  $X = 1$ ). In one example, the candidate BT, TT, or UQTs to be signaled are all vertical splits or horizontal splits depending on previously signaled or derived information. In one example, a first flag is signaled to indicate whether UQT is used. For example, the binarized codewords ordered to represent BT, TT, UQT1, UQT2, UQT3 and UQT4 are  $XX$ ,  $XY$ ,  $YXX$ ,  $YXY$ ,  $YYX$ ,  $YYY$ . In one example, truncated unary code is applied. For example, the binarized codewords ordered to represent BT, TT, UQT1, UQT2, UQT3 and UQT4 are  $X$ ,  $YX$ ,  $YYX$ ,  $YYYX$ ,  $YYYYX$ ,  $YYYYY$ . In one example, a first flag is signaled to indicate whether BT is used. If BT is not used, then a second flag is signaled to indicate whether UQT is used. If UQT is used, which kind of UQT is used is further signaled. For example, the binarized codewords ordered to represent BT, TT, UQT1, UQT2, UQT3 and UQT4 are  $X$ ,  $YX$ ,  $YYXX$ ,  $YYXY$ ,  $YYYX$ ,  $YYYY$ .

(81) In one example, how to signal which kind of split is used in a block may depend on which kinds of split are valid for the block. In the following examples,  $X$  represents 0 or 1 and  $Y = \sim X$  ( $Y = 1$  if  $X = 0$  and  $Y = 0$  if  $X = 1$ ). In one example, the candidate BT, TT or UQTs to be signaled are all vertical splits or horizontal splits depending on previously signaled or derived information. For example, the non-allowed or invalid split cannot be signaled from the encoder to the decoder. Accordingly, there is no codeword to represent the non-allowed or invalid split. In one example, if there is only one kind of split from BT, TT, and UQTs that is valid, then the binarized code to indicate which kind of split (BT, TT, or a kind of UQT) is used is not signaled. In one example, if

there are only two kinds of split from BT, TT, and UQTs that are valid, then a flag is signaled to indicate which one of the two valid splits is used. In one example, the code to indicate which kind of split (BT, TT, or a kind of UQT) is binarized as a truncated unary code. For example, the maximum value of the truncated unary code is  $N-1$ , where  $N$  is the number of valid splits (BT, TT and UQTs). For example, no codeword represents an invalid split. In other words, the invalid split is skipped when building the codeword table. In one example, if no UQT is valid the flag indicating whether UQT is used is not signaled and is inferred to be false. For example, the binarized codewords ordered to represent BT and TT are X and Y. In one example, if only one kind of UQT is valid and UQT is signaled to be used, then no further information is signaled to indicate which UQT is used. The valid UQT is used implicitly. In one example, if only two kinds of UQT are valid and UQT is signaled to be used, then a flag is signaled to indicate which UQT is used. In one example, if only three kinds of UQT are valid and UQT is signaled to be used, then a message is signaled to indicate which UQT is used. For example, the binarized codewords ordered to represent the three UQTs are X, YX, YY. In one example, the binarization and/or signaling method is not changed according to which kinds of split is valid in the block. An invalid split cannot be chosen in a conforming bit-stream.

(82) A bin (bit) of a bin string for indications of split types can be coded by arithmetic coding with one or multiple contexts. In one example, only partial bins of a bin string may be coded with contexts and remaining bins may be coded with bypass mode (e.g., no context is utilized). In an example, all bins of a bin string may be coded with contexts. In an example, all bins of a bin string may be coded with bypass mode. For a bin coded with context, one or multiple contexts may be used.

(83) The context may depend on the position or index of the bin, the partitioning of spatial/temporal neighboring blocks, the current partition depth (e.g., QT depth/BT depth/TT depth/UQT depth/MTT depth) of current block, the partition depth (e.g., QT depth/BT depth/TT depth/UQT depth/MTT depth) of spatial/temporal neighboring blocks and/or spatial/temporal non-adjacent blocks, the coding modes of spatial/temporal neighboring blocks, the width/height of spatial/temporal neighboring blocks, the width/height of the current block, the slice types/picture types/tile group type, the color component, the statistical results of partition types from previously coded blocks, or combinations thereof.

(84) In an example, UQT is not allowed if a split child block crosses more than one Virtual pipeline data units (VPDUs).

(85) In an example, UQT is not allowed if the width/height of the current block satisfies some conditions. For example, the width and height of the current block are  $W$  and  $H$ , and  $T_1$ ,  $T_2$  and  $T$  are some integers. UQT is not allowed if  $W \geq T_1$  and  $H \geq T_2$ . UQT is not allowed if  $W \geq T_1$  or  $H \geq T_2$ . UQT is not allowed if  $W \leq T_1$  and  $H \leq T_2$ . UQT is not allowed if  $W \leq T_1$  or  $H \leq T_2$ . UQT is not allowed if  $W \times H \leq T$ . UQT is not allowed if  $W \times H \geq T$ . Horizontal UQT is not allowed if  $H \leq T$ , for example when  $T=16$ . Horizontal UQT is not allowed if  $H \geq T$ , for example when  $T=128$ . Vertical UQT is not allowed if  $W \leq T$ , for example when  $T=16$ . Vertical UQT is not allowed if  $W \geq T$ , for example when  $T=128$ .  $T_1$ ,  $T_2$ , and  $T$  may be signaled from the encoder to the decoder in a VPS, SPS, PPS, picture header, slice header, tile group header, and/or tile header.  $T_1$ ,  $T_2$ , and  $T$  may depend on color components. For example,  $T_1$ ,  $T_2$ , and  $T$  may be different for luma and chroma components.  $T_1$ ,  $T_2$ , and  $T$  may depend on whether luma coding tree and chroma coding tree are separated. For example,  $T_1$ ,  $T_2$ , and  $T$  may be different for luma and chroma components if the luma coding tree and chroma coding tree are separated. For example, when the transform is not supported for at least one child block due to UQT, UQT split is invalid. For example, when the depth of one block exceeding the allowed depth for UQT splitting, UQT split is invalid. For example, when any of a child block size exceeding the allowed block size due to UQT splitting, UQT split is invalid.

(86) UQT is allowed if the width/height of the current block satisfy some conditions. For example,

the width and height of the current block are  $W$  and  $H$ ,  $T_1$ ,  $T_2$  and  $T$  are some integers. UQT is allowed if  $W \geq T_1$  and  $H \geq T_2$ . UQT is allowed if  $W \geq T_1$  or  $H \geq T_2$ . UQT is allowed if  $WC = T_1$  and  $H \leq T_2$ . UQT is allowed if  $WC = T_1$  or  $H \leq T_2$ . UQT is allowed if  $W \times H \leq T$ . UQT is allowed if  $W \times H \geq T$ . Horizontal UQT is allowed if  $H \leq T$ , for example when  $T = 64$ . Horizontal UQT is allowed if  $H \geq T$ , for example when  $T = 32$ . Vertical UQT is allowed if  $W \leq T$ , for example when  $T = 64$ . Vertical UQT is allowed if  $W \geq T$ , for example when  $T = 32$ .  $T_1$ ,  $T_2$ , and  $T$  may be signaled from the encoder to the decoder in a VPS, SPS, PPS, picture header, slice header, tile group header, and/or tile header.  $T_1$ ,  $T_2$ , and  $T$  may depend on color components. For example,  $T_1$ ,  $T_2$ , and  $T$  may be different for luma and chroma components.  $T_1$ ,  $T_2$ , and  $T$  may depend on whether the luma coding tree and chroma coding tree are separated. For example,  $T_1$ ,  $T_2$ , and  $T$  may be different for luma and chroma components if the luma coding tree and chroma coding tree are separated. In one example,  $T_1$ ,  $T_2$ , and  $T$  may depend on picture/slice types. In one example, Horizontal UQT is allowed if  $TP1 \leq H \leq TP2$  and/or Vertical UQT is allowed if  $TP1 \leq W \leq TP2$  on a P-slice/P-picture or a B-slice/B-picture. Horizontal UQT is allowed if  $TI1 \leq H \leq TI2$  and/or Vertical UQT is allowed if  $TI1 \leq W \leq TI2$  on an I-slice/I-picture. In one example,  $TP1$  is larger than  $TI1$ . For example,  $TI1 = 32$ ,  $TP1 = 64$ ,  $TI2 = TP2 = 64$ . In one example,  $TP2$  is smaller than  $TI2$ . For example,  $TI2 = 64$ ,  $TP2 = 32$ ,  $TI1 = TP1 = 32$ .

(87) UQT is not allowed if the depth of the current block satisfies some conditions. The depth of the current block may refer to QT depth, BT depth, TT depth, UQT depth, and/or MTT depth. UQT is not allowed if the split depth  $\leq T$ . UQT is not allowed if the split depth  $\geq T$ . UQT is not allowed if the QT split depth  $\leq T$ . UQT is not allowed if the QT split depth  $\geq T$ . UQT is not allowed if the BT split depth  $\geq T$ . UQT is not allowed if the BT split depth  $\leq T$ . UQT is not allowed if the TT split depth  $\geq T$ . UQT is not allowed if the TT split depth  $\leq T$ . UQT is not allowed if the UQT split depth  $\leq T$ . UQT is not allowed if the UQT split depth  $\geq T$ . UQT is not allowed if the MTT split depth  $\leq T$ . UQT is not allowed if the MTT split depth  $\geq T$ .  $T$  may be signaled from the encoder to the decoder in a VPS, SPS, PPS, picture header, slice header, tile group header, and/or tile header.  $T$  may depend on color components. For example,  $T_1$ ,  $T_2$ , and  $T$  may be different for the luma and chroma components.  $T$  may depend on whether luma coding tree and chroma coding tree are separated. For example,  $T_1$ ,  $T_2$ , and  $T$  may be different for luma and chroma components if luma coding tree and chroma coding tree are separated.

(88) UQT is allowed if the depth of the current block satisfy some conditions. The depth of the current block may refer to QT depth, BT depth, TT depth, UQT depth, or MTT depth. UQT is allowed if the split depth  $\leq T$ . UQT is allowed if the split depth  $\geq T$ . UQT is allowed if the QT split depth  $\leq T$ . UQT is allowed if the QT split depth  $\geq T$ . UQT is allowed if the BT split depth  $\geq T$ . UQT is allowed if the BT split depth  $\leq T$ . UQT is allowed if the TT split depth  $\geq T$ . UQT is allowed if the TT split depth  $\leq T$ . UQT is allowed if the UQT split depth  $\leq T$ . UQT is allowed if the UQT split depth  $\geq T$ . UQT is allowed if the MTT split depth  $\leq T$ . UQT is allowed if the MTT split depth  $\geq T$ .  $T$  may be signaled from the encoder to the decoder in a VPS, SPS, PPS, picture header, slice header, tile group header, and/or tile header.  $T$  may depend on color components. For example,  $T_1$ ,  $T_2$  and  $T$  may be different for luma and chroma components.  $T$  may depend on whether the luma coding tree and chroma coding tree are separated. For example,  $T_1$ ,  $T_2$ , and  $T$  may be different for luma and chroma components if luma coding tree and chroma coding tree are separated.

(89) In one embodiment, whether and how to use UQT may depend on the position of the current block. For example, whether and how to use UQT may depend on the whether the current block crosses the picture/tile/tile group border or not. In one example, vertical UQT is not allowed if the current block crosses the picture/tile/tile group bottom border.

(90) In one example, horizontal UQT is not allowed if the current block crosses the picture/tile/tile group bottom border. In one example, vertical UQT is not allowed if the current block crosses the picture/tile/tile group right border. In one example, horizontal UQT is not allowed if the current

block crosses the picture/tile/tile group right border. In one example, if a child block split by UQT is totally out of the picture/tile/tile group, the child block may be omitted in the encoding and/or decoding process. In one example, if a child block split by UQT is partially out of the picture/tile/tile group, the following may apply. The part out of the picture may be omitted in the encoding/decoding process. The part inside the picture may be further split. The part inside the picture may be coded as a CU. Whether the part inside the picture is coded as a CU may depend on the width ( $w$ ) and height ( $h$ ) of the part. In one example, the part inside the picture may be coded as a CU if  $w=2nw$ ,  $h=2nh$ , where  $nw$  and  $nh$  are integers. In one example, if any child block split by UQT is partially/fully out of the picture/tile/tile group, UQT is disallowed. In one example, when UQT or certain UQT patterns is disallowed, the signalling of indication of the usage of the patterns is also skipped.

(91) In one example, only UQT1-H, UQT2-H, UQT3-V and UQT4-V (splits **1209**, **1211**, **1205**, and **1207**, respectively) can be used. For example,  $a=c=2$  and  $b=d=4$ . In one example, one bin, possibly with one or more contexts, is signaled to indicate whether a block is split or not, when the block can be non-split, and at least one of QT, BT-H, BT-V, TT-H, TT-V, UQT1-H, UQT2-H, UQT3-V and UQT4-V is applicable in this block. In one example, one bin, possibly with one or more contexts, is signaled to indicate whether a block is split with QT or other splits, when the block is split, and that the block can be QT split, and at least one of BT-H, BT-V, TT-H, TT-V, UQT1-H, UQT2-H, UQT3-V and UQT4-V is applicable in this block. In one example, one bin, possibly with one or more contexts, is signaled to indicate whether a block is split with vertical or horizontal split, when the block is split other than QT, at least one of BT-H, TT-H, UQT1-H, UQT2-H is applicable in this block, and at least one of BT-V, TT-V, UQT3-V and UQT4-V is applicable in this block. In one example, one bin, possibly with one or more contexts, is signaled to indicate whether a block is split with UQT or not, when the block is horizontal split other than QT, and at least one of BT-H, TT-H are available, and at least one of UQT1-H, UQT2-H is applicable in this block. When the block is vertical split other than QT, and at least one of BT-V and TT-V are available, and at least one of UQT3-V, UQT4-V is applicable in this block. In one example, one bin, possibly with one or more contexts, is signaled to indicate whether a block is split with UQT1-H or UQT2-H, when the block is horizontal split with UQT and both UQT1-H, UQT2-H is applicable in this block. UQT3-V or UQT4-V, when the block is vertical split with UQT and both UQT3-V, UQT4-V is applicable in this block. In one example, one or multiple child block of UQT1-H is not allowed to be further split with TT-H. For example, the child block with height equal to  $H/2$  is not allowed to be further split with TT-H. In one example, one or multiple child block of UQT2-H is not allowed to be further split with TT-H. For example, the child block with height equal to  $H/2$  is not allowed to be further split with TT-H. In one example, one or multiple child block of UQT3-V is not allowed to be further split with TT-V. For example, the child block with width equal to  $W/2$  is not allowed to be further split with TT-V. In one example, one or multiple child block of UQT4-V is not allowed to be further split with TT-V. For example, the child block with width equal to  $W/2$  is not allowed to be further split with TT-V. In one example, one or multiple child block of UQT1-H is not allowed to be further split with BT-H. In one example, one or multiple child block of UQT2-H is not allowed to be further split with BT-H. In one example, one or multiple child block of UQT3-V is not allowed to be further split with BT-V. In one example, one or multiple child block of UQT4-V is not allowed to be further split with BT-V. In one example, one or multiple child block of UQT1-H is not allowed to be further split with UQT-H. In one example, one or multiple child block of UQT2-H is not allowed to be further split with UQT-H. In one example, one or multiple child block of UQT3-V is not allowed to be further split with UQT-V. In one example, one or multiple child block of UQT4-V is not allowed to be further split with UQT-V. In one example, one or multiple child block of UQT1-H is not allowed to be further split. In one example, one or multiple child block of UQT2-H is not allowed to be further split. In one example, one or multiple child block of UQT3-V is not allowed to be further split. In one example, one or multiple child

block of UQT4-V is not allowed to be further split. In one example, one of UQT1-H and UQT2-H is not allowed to be further split. In one example, one of UQT1-H and UQT2-H is not allowed to be further horizontal split. In one example, one of UQT3-V and UQT4-V is not allowed to be further split. In one example, one of UQT3-V and UQT4-V is not allowed to be further vertical split. In one example, one or multiple child block of UQT1-H is not allowed to be further split with EQT-H. In one example, one or multiple child block of UQT2-H is not allowed to be further split with EQT-H. In one example, one or multiple child block of UQT3-V is not allowed to be further split with EQT-H. In one example, one or multiple child block of UQT4-V is not allowed to be further split with EQT-H. In one example, one or multiple child block of UQT1-H is not allowed to be further split with EQT-V. In one example, one or multiple child block of UQT2-H is not allowed to be further split with EQT-V. In one example, one or multiple child block of UQT3-V is not allowed to be further split with EQT-V. In one example, one or multiple child block of UQT4-V is not allowed to be further split with EQT-V. The term child block of UQT may refer to a block split from a parent block with UQT, or may refer to a block whose parent block or any ancestor block (e.g. the parent block of the parent block) is split from an ancestor block with UQT.

(92) In one example, only UQT1-H and UQT3-V (splits **1209** and **1205**, respectively) can be used. For example,  $a=2$ . In another example,  $a=c=4$ . In one example, one bin, possibly with one or more contexts, is signaled to indicate whether a block is split or not, when the block can be non-split, and at least one of QT, BT-H, BT-V, TT-H, TT-V, UQT1-H and UQT3-V is applicable in this block. In one example, one bin, possibly with one or more contexts, is signaled to indicate whether a block is split with QT or other splits, when the block is split, and the block can be QT split, and at least one of BT-H, BT-V, TT-H, TT-V, UQT1-H and UQT3-V is applicable in this block. In one example, one bin, possibly with one or more contexts, is signaled to indicate whether a block is split with vertical or horizontal split, when the block is split other than QT, at least one of BT-H, TT-H and UQT1-H is applicable in this block, and at least one of BT-V, TT-V and UQT3-V is applicable in this block. In one example, one bin, possibly with one or more contexts, is signaled to indicate whether a block is split with UQT or not, when the block is horizontal split other than QT, and at least one of BT-H, TT-H are available, and UQT1-H is applicable in this block, and/or when the block is vertical split other than QT, and at least one of BT-V, TT-V are available, and UQT3-V is applicable in this block. In one example, one or multiple child blocks of UQT1-H is not allowed to be further split with TT-H. For example, the child block with height equal to  $H/2$  is not allowed to be further split with TT-H. In one example, one or multiple child block of UQT3-V is not allowed to be further split with TT-V. For example, the child block with width equal to  $W/2$  is not allowed to be further split with TT-V. In one example, one or multiple child block of UQT1-H is not allowed to be further split with BT-H. In one example, one or multiple child block of UQT3-V is not allowed to be further split with BT-V. In one example, one or multiple child block of UQT1-H is not allowed to be further split with UQT-H. In one example, one or multiple child block of UQT3-V is not allowed to be further split with UQT-V.

(93) Indications of splitting types (including EQT, BT, and UQT) may be signaled after the indications of splitting or non-splitting, and/or the indications of QT. In one example, a flag may be signaled to indicate whether the block is split with a first set of UQT and BT or with a second set of UQT and EQT, wherein the UQT patterns in the first and second set are different. For example, one flag may be signaled to indicate whether a certain kind of UQTs is applied or a certain kind of BT is applied on a block. For example, one flag may be signaled to indicate whether UQT2-V/UQT2-H is applied, or BT-V/BT-H is applied on a block. In an example, one flag may be signaled to indicate whether a certain kind of UQTs is applied or a certain kind of EQTs is applied on a block. For example, one flag may be signaled to indicate whether UQT4-V/UQT4-H is applied, or EQT-V/EQT-H is applied on a block. For example, one flag may be signaled to indicate whether UQT2-V/UQT2-H is applied, or EQT-V/EQT-H is applied on a block. In an example, the splitting direction may be further signalled (e.g., horizontal split or vertical split). The above may be utilized

for coding blocks in I slices/pictures. In one example, a flag may be signaled to indicate whether the block is split with BT or with a second set of UQT and EQT. In an example, if the block is not split from BT, another flag may be further signaled to indicate the usage of UQT or EQT. In an example, the splitting direction (e.g., horizontal split or vertical split) may be further signaled. In an example, another flag may be further signaled to indicate the splitting direction (e.g., horizontal split or vertical split). In an example, the splitting pattern (e.g., EQT or UQT) may be further signaled. The above may be utilized for coding blocks in P/B slices/pictures. In an example, the signaling of above-mentioned flags may be skipped when the current block is not split or split according to QT. In an example, only when the current block needs to be further split and/or not split according to QT, the above-mentioned flags may be further signaled.

(94) In one example, whether and/or how to apply the UQT split for a slice/picture may depend on the slice/picture type. In one example, M kinds of UQTs can be applied on a P-slice/P-picture or a B-slice/B-picture, and N kinds of UQTs can be applied on an I-slice/I-picture. In one example, M is smaller than N. For example, M is equal to 2 and N is equal to 4. In one example, M is equal to N, however, the allowed UQTs may be different. For example, UQT2-V/UQT2-H/UQT4-V/UQT4-H can be applied on an I-slice/I-picture. For example, UQT2-V/UQT2-H can be applied on a P-slice/P-picture or a B-slice/B-picture.

(95) Interpretation of the signaled representation of UQT split may depend on the slice/picture type. In one example, the signaled representation of UQT split in an I-slice/I-picture may be different to that in a P-slice/P-picture or a B-slice/B-picture. In one example, one flag may be signaled to indicate whether a certain kind of UQTs is applied or a certain kind of BTs is applied on a block and UQT2-V/UQT2-H/UQT4-V/UQT4-H may be applied on an I-slice/I-picture. In one example, a flag may be signaled to indicate whether the block is split with BT or with a second set of UQT and EQT and UQT2-V/UQT2-H may be applied on a P-slice/P-picture or a B-slice/B-picture.

(96) In one example, whether and/or how to apply UQT split for a slice/picture may depend on the temporal layer of the slice/picture. In one example, UQT is not applied if the temporal layer is larger than a threshold, such as 2. In one example, M kinds of UQTs can be applied on a picture/slice with temporal layer larger than T and N kinds of UQTs can be applied on a picture/slice with temporal layer smaller than or equal to T, wherein M is smaller than N. For example, T is equal to 2, M is equal to 2 and N is equal to 4.

(97) In one example, whether and/or how to apply UQT split for a slice/picture may depend on the whether the slice/picture can be referred by other slices/pictures. In one example, UQT is not applied on a slice/picture if it is not a reference picture for other slices/pictures.

(98) If one kind of UQT is not allowed, the indication for it is not signaled. In one example, if no kind of UQT is allowed, the indication for UQT is not signaled.

(99) How to signal the partitioning method may depend on the picture/slice type. In one example, the flag to indicate whether UQT is applied is signaled before the flag to indicate whether BT/EQT is applied in an I picture/slice, while the flag to indicate whether UQT is applied is signaled after the flag to indicate whether BT/EQT is applied in non-I picture/slice (e.g. a P/B picture/slice). In an example, the flag to indicate whether UQT is applied is signaled after the flag to indicate whether BT/EQT is applied in an I picture/slice, while the flag to indicate whether UQT is applied is signaled before the flag to indicate whether BT/EQT is applied in a non-I picture/slice (e.g. a P/B picture/slice).

(100) In one example, a flag is signaled to indicate the partitioning direction of more than one partitioning methods, including UQT. For example, a flag is signaled to indicate whether vertical or horizontal partitioning is applied for UQT, EQT, and BT.

(101) In one example, the flag to indicate whether UQT is applied (denoted as UQT\_flag) may be coded by arithmetic coding with context model(s). In one example, the context model selection may depend on the picture/slice type (such as I-picture or P/B-picture). In one example, a context

model is used if the current picture is I-picture. Otherwise (not I-picture), a context model is selected from several candidate context models. In one example, the context model selection may depend on the existence of one or multiple neighboring blocks. In one example, the context model selection may depend on dimensions of one or multiple neighboring blocks. In the above examples, the neighboring blocks may include a left neighboring block and/or a above neighboring block. In one example, the context may depend on dimensions of the current block.

(102) FIG. 13 is a schematic diagram 1300 of another example of UQT partitioning structures, for example, for use with AVS-3.0. UQT partitioning is described where a parent CU is split into four CUs asymmetrically. A parent block is split into four sub-blocks with a 1:4:2:1 or a 1:2:4:1 ratio. As shown in diagram 1300, there are 4 splitting types for the UQT partition. With the horizontal UQT, a  $W \times H$  parent CU is divided into two  $W \times H/8$  CUs, one  $W \times H/4$  CU and one  $W \times H/2$  CU as shown by split 1301 and split 1303. A  $W \times H$  parent CU can also be divided into two  $W/8 \times H$  CUs, one  $W/4 \times H$  CU and one  $W/2 \times H$  CU with the vertical UQT as shown in split 1305 and 1307.

(103) FIG. 14 is a schematic diagram of an example LCU partition 1400 with QT, BT, EQT, and UQT partitioning. Different from QT partitioning which cannot be used after BT and EQT partitioning, UQT partitioning can be used after BT and EQT partitioning. UQT can be applied to a direction if the length of the block along the direction is 32 to 64. An example is shown in LCU partition 1400. Similar to BT and EQT partitioning, UQT partitioning is valid after QT/BT/EQT partitioning while its sub-blocks are forbidden to split with QT partition. For B/P slices, the horizontal UQT partitioning can only be applied when the height of current block is 64 and the vertical UQT partitioning can only be applied when the width of current block is 64. Furthermore, only UQT partitioning with 1:2:4:1 ratio is allowed in B/P slices. UQT partitioning is not allowed in the B/P frames not referenced by other frames. One bit is used to signal whether the splitting mode is UQT or not.

(104) FIG. 15 is a schematic diagram 1500 of an example tree type coding structure of an I slice.

FIG. 16 is a schematic diagram 1600 of an example tree type coding structure of a B and/or P slice.

(105) The following is an embodiment of UQT based on AVS-3.0-phase 2.

(106) TABLE-US-00001 coding\_unit\_tree(x0, y0, split, width, height, qt, mode, parent\_split, uqt\_sub\_block,) {  
    isBoundary = ((x0+width) > PicWidthInLuma) || ((y0+height) >  
    PicHeightInLuma)      rightBoundary = ((x0+width) > PicWidthInLuma) && ((y0+height) <=  
    PicHeightInLuma)      bottomBoundary = ( (x0 + width) <= PicWidthInLuma ) && ( (y0 +  
    height) > PicHeightInLuma)      allowNoSplit = 0      allowSplitQt = 0      allowSplitBtVer = 0  
    allowSplitBtHor = 0      allowSplitEqtVer = 0      allowSplitEqtHor = 0  
    allowSplitUqtVer1 = 0      allowSplitUqtHor1 = 0      allowSplitUqtVer2 = 0  
    allowSplitUqtHor2 = 0      if ( isBoundary ) {      allowNoSplit = 0      if  
    ((PictureType == 0) && (width > 64) && (height > 64)) {      allowSplitQt = 1  
        allowNoSplit = 1      }      else if ((width == 64 && height > 64) ||  
    (height == 64 && width > 64)) {      allowSplitBtHor = 1  
        allowSplitBtVer = 1      }      else if (! rightBoundary && ! bottomBoundary) {  
        allowSplitQt = 1      }      else if (rightBoundary) {  
        allowSplitBtVer = 1      }      else if (bottomBoundary) {  
        allowSplitBtHor = 1      }      }      else {      if (((width == 64) && (height > 64)) ||  
    ((height == 64) && (width > 64))) {      allowSplitBtHor = 1  
        allowSplitBtVer = 1      allowNoSplit = 1      }      else if (split >=  
    MaxSplitTimes) {      allowNoSplit = 1      }      else if ((PictureType ==  
    0) && (width == 128) && (height == 128)) {      allowSplitQt = 1  
        allowNoSplit = 1      }      else {      if ((width <= height \* MaxPartRatio)  
    && (height <= width \* MaxPartRatio))      allowNoSplit = 1      if  
    ((width > MinQtSize) && qt)      allowSplitQt = 1      if ((width <=  
    MaxBtSize) && (height <= MaxBtSize) && (width > MinBtSize) && (height <



```

MaxPartRatio*width))                allowSplitBtVer = 1                if ((width <=
MaxBtSize) && (height <= MaxBtSize) && (height > MinBtSize) && (width <
MaxPartRatio*height))                allowSplitBtHor = 1                if ((width <=
MaxEqSize) && (height <= MaxEqSize) && (height >= MinEqSize*2) && (width >=
MinEqSize*4) && (height*4 <= MaxPartRatio*width))                allowSplitEqVer = 1
    if ( (width <= MaxEqSize) && (height <= MaxEqSize) && (width >=
MinEqSize*2) && (height >= MinEqSize*4) && (width*4 <= MaxPartRatio*height) )
        allowSplitEqHor = 1                if (!uqt_disable_flag){
if(PictureType == 0){                if (height == 64 || (height == 32 && width != 64)){
    allowSplitUqtHor1 = 1                allowSplitUqtHor2 = 1
    }                if (weight == 64 || (width == 32 && height != 64)){
        allowSplitUqtVer1 = 1                allowSplitUqtVer2 = 1
    }                }                else if (!uqt_sub_block){
    if (height == 64){                allowSplitUqtHor1 = 1
    if (weight == 64){                allowSplitUqtVer1 = 1
        }                if (uqt_sub_block && (parent_split == 'SPLIT_UQT_VER1' ||
parent_split == 'SPLIT_UQT_HOR1')){                allowSplitEqVer = 0
            allowSplitEqHor = 0                }                }                }
        }                allowSplitBt = allowSplitBtVer || allowSplitBtHor                allowSplitEq =
allowSplitEqVer || allowSplitEqHor                allowSplitFirstUqt = allowSplitUqtVer1 ||
allowSplitUqtHor1                allowSplitSecondUqt = allowSplitUqtVer2 || allowSplitUqtHor2
allowSplitUqt = allowSplitFirstUqt || allowSplitSecondUqt                if (allowSplitQt && (allowNoSplit
|| allowSplitBt || allowSplitEq)) {                qt_split_flag ae(v)                }                else {
QtSplitFlag = allowSplitQt                }                if (! QtSplitFlag) {                if (allowNoSplit &&
(allowSplitBt || allowSplitEq)) {                beut_split_flag ae(v)                }                else {
    BeutSplitFlag = ! allowNoSplit                }                if (BeutSplitFlag) {
    if (PictureType == 0){                if (allowSplitUqt && (allowSplitBt ||
allowSplitEq))                uqt_split_flag ae(v)                if ((UqtSplitFlag &&
allowSplitFirstUqt && allowSplitSecondUqt) || (!UqtSplitFlag && allowSplitBt &&
allowSplitEq))                beut_split_type_flag ae(v)                if ((UqtSplitFlag &&
!BeutSplitTypeFlag && allowSplitUqtHor1 && allowSplitUqtVer1) || (UqtSplitFlag &&
BeutSplitTypeFlag && allowSplitUqtHor2 && allowSplitUqtVer2) || (!UqtSplitFlag &&
!BeutSplitTypeFlag && allowSplitBtHor && allowSplitBtVer) || (!UqtSplitFlag &&
BeutSplitTypeFlag && allowSplitEqHor && allowSplitEqVer))
    beut_split_dir_flag ae(v)                }                else {                if (allowSplitBt
&& (allowSplitEq || allowSplitFirstUqt))                beut_split_type_flag ae(v)
    if ((BeutSplitTypeFlag && allowSplitEq && allowSplitFirstUqt) ||
(!BeutSplitTypeFlag && allowSplitEqHor && allowSplitEqVer))
    uqt_split_flag ae(v)                if ((BeutSplitTypeFlag && UqtSplitFlag &&
allowSplitUqtHor1 && allowSplitUqtVer1) || (BeutSplitTypeFlag && !UqtSplitFlag &&
allowSplitEqHor && allowSplitEqVer) || (!BeutSplitTypeFlag && allowSplitBtHor &&
allowSplitBtVer))
    beut_split_dir_flag ae(v)                }
custom character                custom character custom character
custom character custom character custom character custom character
custom character custom character                custom character custom character
    }                }                if ((PictureType != 0) && (((BeutSplitFlag && ! BeutSplitTypeFlag) ||
QtSplitFlag) && (width * height == 64)) || (BeutSplitTypeFlag && (width * height == 128)))) {
        root_cu_mode ae(v)                modeChild = root_cu_mode ? 'PRED_Intra_Only' :
'PRED_Inter_Only'                }                else {                modeChild = mode                }                if
(ChildSizeOccur4) {                if (Component == 0) {                Luma Width = width

```

```

        LumaHeight = height                                Component = 1                                }                                }                                if
(parent_split == 'SPLIT_UQT_VER1' || parent_split == 'SPLIT_UQT_VER2' || parent_split ==
'SPLIT_UQT_HOR1' || parent_split == 'SPLIT_UQT_HOR2'){                                uqt_sub_block_child = 1
    }                                else {                                uqt_sub_block_child = uqt_sub_block                                }                                if (BlockSplitMode ==
'SPLIT_QT') {                                QtWidth = width / 2                                QtHeight = height / 2                                x1 = x0 +
QtWidth                                y1 = y0 + QtHeight                                coding_unit_tree(x0, y0, split+1, QtWidth,
QtHeight, 1, modeChild, BlockSplitMode, uqt_sub_block_child)                                if (x1 <
PicWidthInLuma)                                coding_unit_tree(x1, y0, split+1, QtWidth, QtHeight, 1,
modeChild, BlockSplitMode, uqt_sub_block_child)                                if (y1 < PicHeightInLuma)
coding_unit_tree(x0, y1, split+1, QtWidth, QtHeight, 1, modeChild,
BlockSplitMode, uqt_sub_block_child)                                if ((x1 < PicWidthInLuma) && (y1 <
PicHeightInLuma))                                coding_unit_tree(x1, y1, split+1, QtWidth, QtHeight, 1,
modeChild, BlockSplitMode, uqt_sub_block_child)                                if ((LumaWidth == width) &&
(LumaHeight = height) && ChildSizeOccur4) {                                coding_unit(x0, y0, width, height,
'PRED_No_Constraint', 'COMPONENT_Chroma')                                Component = 0                                }
    }                                else if (BlockSplitMode == 'SPLIT_BT_VER') {                                x1 = x0 + width / 2
coding_unit_tree(x0, y0, split+1, width/2, height, 0, modeChild, BlockSplitMode,
uqt_sub_block_child)                                if (x1 < PicWidthInLuma)                                coding_unit_tree(x1, y0,
split+1, width/2, height, 0, modeChild, BlockSplitMode, uqt_sub_block_child)                                if
((LumaWidth == width) && (LumaHeight = height) && ChildSizeOccur4) {
coding_unit(x0, y0, width, height, 'PRED_No_Constraint', 'COMPONENT_Chroma')
Component = 0                                }                                }                                else if (BlockSplitMode ==
'SPLIT_BT_HOR') {                                y1 = y0 + height / 2                                coding_unit_tree(x0, y0, split+1,
width, height/2, 0, modeChild, BlockSplitMode, uqt_sub_block_child)                                if (y1 <
PicHeightInLuma)                                coding_unit_tree(x0, y1, split+1, width, height/2, 0, modeChild,
BlockSplitMode, uqt_sub_block_child)                                if ((LumaWidth == width) && (LumaHeight =
height) && ChildSizeOccur4) {                                coding_unit(x0, y0, width, height,
'PRED_No_Constraint', 'COMPONENT_Chroma')                                Component = 0                                }
    }                                else if (BlockSplitMode == 'SPLIT_EQT_VER') {                                x1 = x0 + width / 4
x2 = x0 + (3 * width / 4)                                y1 = y0 + height / 2                                coding_unit_tree(x0,
y0, split+1, width/4, height, 0, modeChild, BlockSplitMode, uqt_sub_block_child)
coding_unit_tree(x1, y0, split+1, width/2, height/2, 0, modeChild, BlockSplitMode,
uqt_sub_block_child)                                coding_unit_tree(x1, y1, split+1, width/2, height/2, 0, modeChild,
BlockSplitMode, uqt_sub_block_child)                                coding_unit_tree(x2, y0, split+ 1, width/4,
height, 0, modeChild, BlockSplitMode, uqt_sub_block_child)                                if ((Luma Width = width)
&& (LumaHeight = height) && ChildSizeOccur4) {                                coding_unit(x0, y0, width,
height, 'PRED_No_Constraint', 'COMPONENT_Chroma')                                Component = 0
    }                                }                                else if (BlockSplitMode == 'SPLIT_EQT_HOR') {                                x1 = x0 +
width / 2                                y1 = y0 + height / 4                                y2 = y0 + (3 * height / 4)
coding_unit_tree(x0, y0, split+1, width, height/4, 0, modeChild, BlockSplitMode,
uqt_sub_block_child)                                coding_unit_tree(x0, y1, split+1, width/2, height/2, 0, modeChild,
BlockSplitMode, uqt_sub_block_child)                                coding_unit_tree(x1, y1, split+1, width/2,
height/2, 0, modeChild, BlockSplitMode, uqt_sub_block_child)                                coding_unit_tree(x0, y2,
split+1, width, height/4, 0, modeChild, BlockSplitMode, uqt_sub_block_child)                                if
((LumaWidth == width) && (LumaHeight = height) && ChildSizeOccur4) {
coding_unit(x0, y0, width, height, 'PRED_No_Constraint', 'COMPONENT_Chroma')
Component = 0                                }                                }                                else if (BlockSplitMode ==
'SPLIT_UQT_HOR1' | BlockSplitMode == 'SPLIT_UQT_HOR2' || BlockSplitMode =
'SPLIT_UQT_VER1' = || BlockSplitMode == 'SPLIT_UQT_VER2') {                                uqt_idx =
BlockSplitMode == 'SPLIT_UQT_HOR1' ? 0 : (BlockSplitMode == 'SPLIT_UQT_HOR2' ? 1 :

```

```

(BlockSplitMode == 'SPLIT_UQT_VER1' ? 2 : 3))      x_tab[4][4] = {[0, 0, 0, 0], [0, 0, 0,
0], [0, 1/8, 1/2, 1/4], [0, 1/8, 1/4, 1/2]}      y_tab[4][4] = {[0, 1/8, 1/2, 1/4], [0, 1/8, 1/4, 1/2],
[0, 0, 0, 0], [0, 0, 0, 0]}      w_tab[4][4] = {[0, 0, 0, 0], [0, 0, 0, 0], [1/8, 1/2, 1/4, 1/8], [1/8,
1/4, 1/2, 1/8]}      h_tab[4][4] = {[1/8, 1/2, 1/4, 1/8], [1/8, 1/4, 1/2, 1/8], [0, 0, 0, 0], [0, 0, 0,
0]}      coding_unit_tree(x0+x_tab[uqt_idx][0], y0+y_tab[uqt_idx][0], split+1,
width*w_tab[uqt_idx][0], height*h_tab[uqt_idx][0], 0, modeChild, BlockSplitMode,
uqt_sub_block_child)      coding_unit_tree(x0+x_tab[uqt_idx][1], y0+y_tab[uqt_idx][1],
split+1, width*w_tab[uqt_idx][1], height*h_tab[uqt_idx][1], 0, modeChild, BlockSplitMode,
uqt_sub_block_child)      coding_unit_tree(x0+x_tab[uqt_idx][2], y0+y_tab[uqt_idx][2],
split+1, width*w_tab[uqt_idx][2], height*h_tab[uqt_idx][2], 0, modeChild, BlockSplitMode,
uqt_sub_block_child)      coding_unit_tree(x0+x_tab[uqt_idx][3], y0+y_tab[uqt_idx][3],
split+1, width*w_tab[uqt_idx][3], height*h_tab[uqt_idx][3], 0, modeChild, BlockSplitMode,
uqt_sub_block_child)      if ((Luma Width == width) && (LumaHeight = height) &&
ChildSizeOccur4) {      coding_unit(x0, y0, width, height, 'PRED_No_Constraint',
'COMPONENT_Chroma')      Component = 0      }      else {
if (Component == 0) {      coding_unit(x0, y0, width, height, mode,
'COMPONENT_LUMACHROMA')      }      else if (Component == 1) {
coding_unit(x0, y0, width, height, mode, 'COMPONENT_LUMA')      }
} }

```

(107) The following is an embodiment of context derivation. A variable ctxIdxInc is defined to indicate the context model for uqt\_split\_flag. Block A represents a left neighboring block with dimensions  $W_a \times H_a$ . Block B represents an above neighboring block with dimensions  $W_b \times H_b$ . Block E represents the current block with dimensions  $W_e \times H_e$ . ctxIdxInc is derived as below:

(108) TABLE-US-00002 If the current block is I-picture, ctxIdxInc is set equal to 0. Otherwise, ctxIdxInc is derived as below: If (Block A exists and  $H_a < H_e$ ) and (Block B exists and  $W_b < W_e$ ), then ctxIdxInc is set equal to 2. Otherwise, If (Block A exists and  $H_a < H_e$ ) or (Block B exists and  $W_b < W_e$ ), then ctxIdxInc is set equal to 1.

(109) Then ctxIdxInc is further revised as below:

(110) TABLE-US-00003 If  $W_e \times H_e > 1024$ , ctxIdxInc is unchanged; Otherwise, if  $W_e \times H_e > 256$ , ctxIdxInc is increased by 3; Otherwise, ctxIdxInc is increased by 6.

(111) FIG. 17 is a schematic diagram 1700 of example ETT partitioning structures, including an ETT-V split 1701 and an ETT-H split 1703. When employing ETT, a block with dimensions width times height ( $W \times H$ ) is split into three partitions with dimensions  $W_1 \times H_1$ ,  $W_2 \times H_2$ , and  $W_3 \times H_3$ .  $W_1$ ,  $W_2$ ,  $W_3$ ,  $H_1$ ,  $H_2$ ,  $H_3$  are all integers. In an example, and at least one of the parameters is not in the form of power of 2.  $W_1$ ,  $W_2$ , and  $W_3$  are widths of resulting sub-blocks.  $H_1$ ,  $H_2$ , and  $H_3$  are heights of resulting sub-blocks. In one example,  $W_2$  cannot be in a form of  $W_2 = 2N_2$  with any positive integer  $N_2$ . In another example,  $H_2$  cannot be in a form of  $H_2 = 2 \cdot \text{sup} \cdot N_2$  with any positive integer  $N_2$ . In one example, at least one of the parameters is in the form of power of 2. In one example,  $W_1$  is in a form of  $W_1 = 2 \cdot \text{sup} \cdot N_1$  with a positive integer  $N_1$ . In another example,  $H_1$  is in a form of  $H_1 = 2 \cdot \text{sup} \cdot N_1$  with a positive integer  $N_1$ .

(112) In one example, ETT only splits one partition in a vertical direction, for example where  $W_1 = a_1 \cdot W$ ,  $W_2 = a_2 \cdot W$ , and  $W_3 = a_3 \cdot W$ , where  $a_1 + a_2 + a_3 = 1$ , and where  $H_1 = H_2 = H_3 = H$ . This kind of ETT is vertical split and may be referred to as ETT-V. In one example, ETT-V split 1701 can be used where  $W_1 = W/8$ ,  $W_2 = 3 \cdot W/4$ ,  $W_3 = W/8$ , and  $H_1 = H_2 = H_3 = H$ . In one example, ETT only splits one partition in horizontal direction, for example where  $H_1 = a_1 \cdot H$ ,  $H_2 = a_2 \cdot H$ , and  $H_3 = a_3 \cdot H$ , where  $a_1 + a_2 + a_3 = 1$ , and where  $W_1 = W_2 = W_3 = W$ . This kind of ETT is a horizontal split and may be referred to as ETT-H. In one example, ETT-H split 1703 can be used where  $H_1 = H/8$ ,  $H_2 = 3 \cdot H/4$ ,  $H_3 = H/8$ , and  $W_1 = W_2 = W_3 = W$ .

(113) The following are example technical problems solved by disclosed technical solutions.

QT/BT/TT/UQT can only split a block into sub-blocks with dyadic dimensions. Dyadic dimensions

describe a case where the width and height of a sub-block must be in a form  $a \cdot 2^{\text{sup}.N}$ , wherein N is a positive integer.

(114) Disclosed herein are mechanisms to address one or more of the problems listed above. Unsymmetric Binary Tree (UBT) partitioning can be employed to split blocks into non-dyadic dimensions. In addition, non-dyadic sub-blocks created by UBT partitioning can be further partitioned. A dyadic block is a block with only dimensions that can be expressed as a power of two (2, 4, 8, 16, 32, 64, 128, etc.) A non-dyadic block is any block with at least one dimension that is not a power of two. Most numbers are not a power of two, and hence UBT partitioning can be applied to a large set of blocks that cannot be partitioned by QT, BT, TT, and/or UQT. A UBT partition may also be referred to as a UBT split. A UBT split can be applied to a parent block to results in two sub-blocks that may (or may not) have different dimensions. For example, the UBT split may result in two sub-blocks where at least one of the sub-blocks has a side with a size that is not a power of two. For example, when a parent block has a width (W) and height (H), the resulting first sub-block has dimensions of width (W1) and height (H1) and the resulting second sub-block has dimensions of width (W2) and height (H2), where at least one of W1, W2, H1, and H2 is non-dyadic. A block with a non-dyadic dimension can be referred to as a non-dyadic block. The UBT partition can be a vertical UBT (UBT-V) partition and/or a horizontal UBT (UBT-H) partition. Type 0 UBT occurs when the first sub-block has a side that is less than half the size of the corresponding side of the parent block. Type 1 UBT occurs when the first sub-block has a side that is more than half (but not equal to) the size of the corresponding side of the parent block.

(115) In an example, the non-dyadic, such as a sub-block resulting from a UBT split, can be further split. For example, a horizontal split, such as BT, TT, UQT, ETT, or UBT can be applied to a block with a non-dyadic width and a dyadic height. Vertical splits can also be applied resulting in sub-blocks with widths that can be dyadic, non-dyadic, or combinations thereof. In another example, a vertical split, such as BT, TT, UQT, ETT, or UBT can be applied to a block with a non-dyadic height and a dyadic width. Horizontal splits can also be applied resulting in sub-blocks with heights that can be dyadic, non-dyadic, or combinations thereof. Vertical or horizontal splits can also be applied to a block with a non-dyadic width and a non-dyadic height, which can result in sub-blocks with heights that can be dyadic, non-dyadic, or combinations thereof. In an example, a UBT split cannot be applied to a sub-block that is created as a result of a UBT split. Further, different constraints can be applied to dyadic blocks and non-dyadic blocks. For example, a tree depth may be maintained for a tree structure that splits a block into sub-blocks. Splits to dyadic blocks may increase the tree depth by one, while splits to non-dyadic blocks may increase the tree depth by more than one for sub-blocks that are less than half the size of the parent block. In an example, horizontal or vertical splits may be disallowed after UBT-V or UBT-H splits. In an example, a syntax element, such as a flag or index, can be used to signal UBT, vertical/horizontal UBT, type 0/type 1 UBT, UBT split ratio, or combinations thereof in a bitstream.

(116) FIG. 18 is a schematic diagram 1800 of example  $\frac{1}{4}$  UBT partitioning structures, which includes vertical UBT (UBT-V) partitions and horizontal UBT (UBT-H) partitions. In the examples shown, a parent block is split into two sub-blocks. One sub-block has a side that is  $\frac{1}{4}$  of the dimension of the split side of the parent and one sub-block has a side that is  $\frac{3}{4}$  of the dimension of the split side of the parent. The schematic diagram 1800 includes UBT-V split 1801, a UBT-V split 1803, a UBT-H split 1805, and a UBT-H split 1807. The sub-blocks are created with different indices. For example, the sub-blocks on the left or top have a lower index (e.g., zero) than the sub-blocks on the right or bottom. Accordingly, the sub-blocks on the left or top can be referred to as a first sub-block and the sub-blocks on the right or bottom can be referred to as a second sub-block. A UBT split may be referred to as a type 0 UBT when the first block is less than half the size of the parent block and/or a type 1 UBT when the first block is greater than half the size of the parent block. Accordingly, UBT-V split 1801 and UBT-H split 1805 are type 0 splits and UBT-V split 1803 and UBT-H split 1807 are type 1 splits. The coding of UBT-V splits is left to right and the

coding of UBT-H splits is top to bottom.

(117) For example, a parent block can have a width (W) and a height (H). The first sub-block (left or top) can have a width (W1) and height (H1). The second sub-block (right or bottom) can have a width (W2) and height (H2). In a UBT-V split,  $W1=a \times W$ ,  $W2=(1-a) \times W$ , and  $H1=H2=H$  where  $a$  is a partition ratio, for example in the form of a fraction. In a UBT-H split,  $H1=a \times H$ ,  $H2=(1-a) \times H$ , and  $W1=W2=W$ . In a type 0 split,  $a$  is smaller than one half. In a type 1 split,  $a$  is smaller than one half. In UBT-V split **1801** and UBT-H split **1805**,  $a$  is  $\frac{1}{4}$ . In UBT-V split **1803** and UBT-H split **1807**,  $a$  is  $\frac{3}{4}$ .

(118) One of the benefits of UBT is that a UBT partition can be used to create a non-dyadic sub-block. A non-dyadic block (or sub-block) is a block with at least one dimension that cannot be expressed as a power of two (e.g., there is no positive integer value of  $N$  where the dimension equals  $2^{\text{sup}.N}$ ). A dyadic sub-block is a block where all dimensions can be expressed as a power of two (all sides can be expressed by  $2^{\text{sup}.N}$  where  $N$  is a positive integer). Accordingly, the first sub-block, the second sub-block, or both may have a non-dyadic dimension and hence may be a non-dyadic sub-block.

(119) FIG. **19** is a schematic diagram **1900** of example  $\frac{1}{8}$  UBT partitioning structures, which includes UBT-V partitions and UBT-H partitions. In the examples shown, a parent block is split into two sub-blocks. One sub-block has a side that is  $\frac{1}{8}$  of the dimension of the split side of the parent and one sub-block has a side that is  $\frac{7}{8}$  of the dimension of the split side of the parent. The schematic diagram **1900** includes UBT-V split **1901**, a UBT-V split **1903**, a UBT-H split **1905**, and a UBT-H split **1907**. In UBT-V split **1901** and UBT-H split **1905**, the first sub-block (left or top) is smaller than one half, and hence such splits are type 0 splits. In UBT-V split **1903** and UBT-H split **1907**, the first sub-block (left or top) is larger than one half, and hence such splits are type 1 splits. For example, a parent block can have a width (W) and a height (H). The first sub-block (left or top) can have a width (W1) and height (H1). The second sub-block (right or bottom) can have a width (W2) and height (H2). In a UBT-V split,  $W1=a \times W$ ,  $W2=(1-a) \times W$ , and  $H1=H2=H$  where  $a$  is a partition ratio, for example in the form of a fraction. In a UBT-H split,  $H1=a \times H$ ,  $H2=(1-a) \times H$ , and  $W1=W2=W$ . In a type 0 split,  $a$  is smaller than one half. In a type 1 split,  $a$  is smaller than one half. In UBT-V split **1901** and UBT-H split **1905**,  $a$  is  $\frac{1}{8}$ . In UBT-V split **1903** and UBT-H split **1907**,  $a$  is  $\frac{7}{8}$ . One of the benefits of UBT is that a UBT partition can be used to create a non-dyadic sub-block. As described above, the first sub-block, the second sub-block, or both may have a non-dyadic dimension and hence may be a non-dyadic sub-block.

(120) FIG. **20** is a schematic diagram **2000** of example  $\frac{1}{16}$  UBT partitioning structures, which includes UBT-V partitions and UBT-H partitions. In the examples shown, a parent block is split into two sub-blocks. One sub-block has a side that is  $\frac{1}{16}$  of the dimension of the split side of the parent and one sub-block has a side that is  $\frac{15}{16}$  of the dimension of the split side of the parent. The schematic diagram **2000** includes UBT-V split **2001**, a UBT-V split **2003**, a UBT-H split **2005**, and a UBT-H split **2007**. In UBT-V split **2001** and UBT-H split **2005**, the first sub-block (left or top) is smaller than one half, and hence such splits are type 0 splits. In UBT-V split **2003** and UBT-H split **2007**, the first sub-block (left or top) is larger than one half, and hence such splits are type 1 splits. For example, a parent block can have a width (W) and a height (H). The first sub-block (left or top) can have a width (W1) and height (H1). The second sub-block (right or bottom) can have a width (W2) and height (H2). In a UBT-V split,  $W1=a \times W$ ,  $W2=(1-a) \times W$ , and  $H1=H2=H$  where  $a$  is a partition ratio, for example in the form of a fraction. In a UBT-H split,  $H1=a \times H$ ,  $H2=(1-a) \times H$ , and  $W1=W2=W$ . In a type 0 split,  $a$  is smaller than one half. In a type 1 split,  $a$  is smaller than one half. In UBT-V split **2001** and UBT-H split **2005**,  $a$  is  $\frac{1}{16}$ . In UBT-V split **2003** and UBT-H split **2007**,  $a$  is  $\frac{15}{16}$ . As described above, the first sub-block, the second sub-block, or both may have a non-dyadic dimension and hence may be a non-dyadic sub-block.

(121) FIGS. **21-22** are schematic diagrams of example non-dyadic partitions. FIG. **21** includes schematic diagram **2100**, which depicts split **2101**, split **2103**, split **2105**, split **2107**, split **2109**,



$\log_2 H$ , and  $H_1 = 1 \leq N$ , and  $H_2 = H - H_1$ , where  $N < 2^{\lceil \log_2 H \rceil}$ . In one example,  $H_2 = 1 \leq N$ , and  $H_1 = H - H_2$ , where  $N < 2^{\lceil \log_2 H \rceil}$ . In some examples, horizontal TT, ETT, QT, EQT, or UQT split may be applied to the  $W \times H$  block.

(126) Accordingly, to address the problems mentioned above, several methods are disclosed to handle the issues caused by QT/BT/TT/UQT as discussed above. The methods result in achieving better coding performance.

(127) The detailed embodiments below should be considered as examples to explain general concepts. These embodiments should not be interpreted in a narrow way. Furthermore, these embodiments can be combined in any manner. In the following discussion, QT, BT, TT, UQT, and ETT may refer to QT split, BT split, TT split, UQT split and ETT split, respectively. In the following discussion, a block is a dyadic block if both width and height is a dyadic number, which is in a form of a  $2^N$  with  $N$  being a positive integer. In the following discussion, a block is a non-dyadic block if at least one of width and height is a non-dyadic number, which cannot be represented in a form of a  $2^N$  with  $N$  being a positive integer. In the following discussion, split and partitioning have the same meaning.

#### Example 1

(128) A block of dimensions  $W \times H$  can be split into two sub-blocks dimensions  $W_1 \times H_1$  and  $W_2 \times H_2$ , wherein the two sub-blocks may have different dimensions. Such a split is named as “Unsymmetric Binary Tree” (UBT) split.

#### Example 2

(129) In one example, at least one of the  $W_1$ ,  $H_1$ ,  $W_2$ , and  $H_2$  is non-dyadic, where a non-dyadic is a block with a dimension that is not in the form of a power of two. In one example, one of the subblocks is a dyadic block and the other is a non-dyadic block.

#### Example 3

(130) In one example,  $W_1 = a \times W$ ,  $W_2 = (1-a) \times W$ , and  $H_1 = H_2 = H$ . In such as case, the partition may be called a vertical UBT (UBT-V). In one example,  $a$  may be smaller than  $1/2$ , such as  $1/4$ ,  $1/8$ ,  $1/16$ ,  $1/32$ ,  $1/64$ , etc. In such a case, the partition may be called a Type 0 UBT-V. In one example,  $a$  may be larger than  $1/2$ , such as  $3/4$ ,  $7/8$ ,  $15/16$ ,  $31/32$ ,  $63/64$ , etc. In such a case, the partition is called a Type 1 UBT-V. In one example, the coding order of the two sub-blocks is from left to right.

#### Example 4

(131) In one example,  $H_1 = a \times H$ ,  $H_2 = (1-a) \times H$ ,  $W_1 = W_2 = W$ . In such as case, the partition may be called a horizontal UBT (UBT-H). In one example,  $a$  may be smaller than  $1/2$ , such as  $1/4$ ,  $1/8$ ,  $1/16$ ,  $1/32$ ,  $1/64$ , etc. In such a case, the partition is called a Type 0 UBT-H. In one example,  $a$  may be larger than  $1/2$ , such as  $3/4$ ,  $7/8$ ,  $15/16$ ,  $31/32$ ,  $63/64$ , etc. In such a case, the partition may be called a Type 1 UBT-H. In one example, the coding order of the two sub-blocks is from top to bottom.

#### Example 5

(132) A non-dyadic block with dimensions  $W \times H$  (wherein at least one of  $W$  and  $H$  is not in the form of power of 2) may be further split into subblocks.

#### Example 6

(133) In one example, the following examples may apply when  $W$  is a non-dyadic number but  $H$  is a dyadic number. For example, a horizontal-split method which can be used for a dyadic block, such as horizontal-BT, horizontal-TT, horizontal-UQT, horizontal-ETT, and/or horizontal-UBT can be applied to the  $W \times H$  block in the same manner, as shown by split **2101**, split **2103**, split **2105**, split **2107**, and split **2109**.

(134) For example, a vertical split can be applied to the  $W \times H$  block. In one example, the  $W \times H$  block may be split into two subblocks with dimensions  $W_1 \times H$  and  $W_2 \times H$ , as shown by split **2111**, split **2113**, and split **2115**. In one example, at least one of  $W_1$  and  $W_2$  is a dyadic number. In one example, both  $W_1$  and  $W_2$  are dyadic numbers. In one example, both  $W_1$  and  $W_2$  are non-dyadic numbers. In one example,  $W_1 = W_2 = W/2$ . In one example,  $W_1 = 2^{\lceil \log_2 W \rceil}$ , and

$W2=W-W1$ . In one example,  $W2=\sup.1<< \lfloor \log.\sup.2 .\sup.W \rfloor$ , and  $W1=W-W2$ . In one example,  $W1=\sup.1<< \lceil \log.\sup.2 .\sup.W \rceil$ , and  $W2=W-W1$ . In one example,  $W2=\sup.1<< \lceil \log.\sup.2 .\sup.W \rceil$ , and  $W1=W-W2$ . In one example,  $W1=1<<N$ , and  $W2=W-W1$ , where  $N<\sup.1<< \lfloor \log.\sup.2 .\sup.W \rfloor$ . In one example,  $W2=1<<N$ , and  $W1=W-W2$ , where  $N<\sup.1<< \lfloor \log.\sup.2 .\sup.W \rfloor$ . In some examples, vertical TT, ETT, QT, EQT, or UQT split may be applied to the  $W \times H$  block.

#### Example 7

(135) In one example, the following examples may apply when  $H$  is a non-dyadic number but  $W$  is a dyadic number. For example, a vertical-split method which can be used for a dyadic block, such as vertical-BT, vertical-TT, vertical-UQT, vertical-ETT, and/or vertical-UBT can be applied to the  $W \times H$  block in the same manner, as shown by split **2201**, split **2203**, split **2205**, split **2207**, and split **2209**.

(136) For example, a horizontal split can be applied to the  $W \times H$  block. In one example, the  $W \times H$  block may be split into two subblocks with dimensions  $W \times H1$  and  $W \times H2$ , as shown by split **2211**, split **2213**, and split **2215**. In one example, at least one of  $H1$  and  $H2$  is a dyadic number. In one example, both  $H1$  and  $H2$  are dyadic numbers. In one example, both  $H1$  and  $H2$  are non-dyadic numbers. In one example,  $H1=H2=H/2$ . In one example,  $H1=\sup.1<< \lfloor \log.\sup.2 .\sup.H \rfloor$ , and  $H2=H-H1$ . In one example,  $H2=\sup.1<< \lfloor \log.\sup.2 .\sup.H \rfloor$ , and  $H1=H-H2$ . In one example,  $H1=\sup.1<< \lceil \log.\sup.2 .\sup.H \rceil$ , and  $H2=H-H1$ . In one example,  $H2=\sup.1<< \lceil \log.\sup.2 .\sup.H \rceil$ , and  $H1=H-H2$ . In one example,  $H1=1<<N$ , and  $H2=H-H1$ , where  $N<\sup.1<< \lfloor \log.\sup.2 .\sup.H \rfloor$ . In one example,  $H2=1<<N$ , and  $H1=H-H2$ , where  $N<\sup.1<< \lfloor \log.\sup.2 .\sup.H \rfloor$ . In some examples, horizontal TT, ETT, QT, EQT, or UQT split may be applied to the  $W \times H$  block.

#### Example 8

(137) In one example, the following examples may apply when both  $W$  and  $H$  are non-dyadic numbers. For example, a vertical split can be applied to the  $W \times H$  block. In one example, the  $W \times H$  block may be split into two subblocks with dimensions  $W1 \times H$  and  $W2 \times H$ , as shown by split **2111**, split **2113**, and split **2115**.

(138) In one example, at least one of  $W1$  and  $W2$  is a dyadic number. In one example, both  $W1$  and  $W2$  are dyadic numbers. In one example, both  $W1$  and  $W2$  are non-dyadic numbers. In one example,  $W1=W2=W/2$ . In one example,  $W1=\sup.1<< \lfloor \log.\sup.2 .\sup.W \rfloor$ , and  $W2=W-W1$ . In one example,  $W2=\sup.1<< \lfloor \log.\sup.2 .\sup.W \rfloor$ , and  $W1=W-W2$ . In one example,  $W1=\sup.1<< \lceil \log.\sup.2 .\sup.W \rceil$ , and  $W2=W-W1$ . In one example,  $W2=\sup.1<< \lceil \log.\sup.2 .\sup.W \rceil$ , and  $W1=W-W2$ . In one example,  $W1=1<<N$ , and  $W2=W-W1$ , where  $N<\sup.1<< \lfloor \log.\sup.2 .\sup.W \rfloor$ . In one example,  $W2=1<<N$ , and  $W1=W-W2$ , where  $N<\sup.1<< \lfloor \log.\sup.2 .\sup.W \rfloor$ .

(139) For example, a horizontal split can be applied to the  $W \times H$  block. In one example, the  $W \times H$  block may be split into two subblocks with dimensions  $W \times H1$  and  $W \times H2$ , as shown by split **2211**, split **2213**, and split **2215**. In one example, at least one of  $H1$  and  $H2$  is a dyadic number. In one example, both  $H1$  and  $H2$  are dyadic numbers. In one example, both  $H1$  and  $H2$  are non-dyadic numbers. In one example,  $H1=H2=H/2$ . In one example,  $H1=\sup.1<< \lfloor \log.\sup.2 .\sup.H \rfloor$ , and  $H2=H-H1$ . In one example,  $H2=\sup.1<< \lfloor \log.\sup.2 .\sup.H \rfloor$ , and  $H1=H-H2$ . In one example,  $H1=\sup.1<< \lceil \log.\sup.2 .\sup.H \rceil$ , and  $H2=H-H1$ . In one example,  $H2=\sup.1<< \lceil \log.\sup.2 .\sup.H \rceil$ , and  $H1=H-H2$ . In one example,  $H1=1<<N$ , and  $H2=H-H1$ , where  $N<\sup.1<< \lfloor \log.\sup.2 .\sup.H \rfloor$ . In one example,  $H2=1<<N$ , and  $H1=H-H2$ , where  $N<\sup.1<< \lfloor \log.\sup.2 .\sup.H \rfloor$ . In some examples, vertical or horizontal TT, ETT, QT, EQT, or UQT split may be applied to the  $W \times H$  block.

#### Example 9

(140) In an example, a rule may disallow a sub-block split from UBT from receiving a further split according to UBT. In an example, the indication of the further UBT split for the sub-block is not



signalled.

#### Example 10

(141) In one example, different constraints or rules on block split may be applied on dyadic blocks and non-dyadic blocks.

#### Example 11

(142) In an example, the tree-depth of a block split by UBT may be added by a number that is different to the number when the block is split by a BT. For example, the multiple-tree depth (MT-depth) or binary-tree depth (BT-depth) of a sub-block may be derived as  $D + \delta$ , where  $D$  is the MT-depth of the current block to be split by UBT and  $\delta$  is an integer. For example,  $\delta > 1$  when the sub-block is smaller than half of the current block. For example,  $\delta = 2$  when the sub-block is  $\frac{1}{4}$  of the current block. Further,  $\delta = 3$  when the sub-block is  $\frac{1}{8}$  of the current block. For example,  $\delta \leq 1$  when the sub-block is smaller than half of the current block. For example,  $\delta = 1$  or  $0$  if the sub-block is  $\frac{3}{4}$  of the current block.

#### Example 12

(143) In one example, when the current block is split from a parent block by UBT-V, then the current block is not allowed to be further split by one, multiple, all vertical splits such as BT-V, TT-V, UQT-V, UBT-V, ETT-V, EQT-V. In one example, when the current block is split from a parent block by UBT-H, then the current block is not allowed to be further split by one or multiple or all vertical splits such as BT-V, TT-V, UQT-V, UBT-V, ETT-V, EQT-V. In one example, when the current block is split from a parent block by UBT-V, then the current block is not allowed to be further split by one or multiple or all horizontal splits such as BT-H, TT-H, UQT-H, UBT-H, ETT-H, EQT-H. In one example, when the current block is split from a parent block by UBT-H, then the current block is not allowed to be further split by one or multiple or all horizontal splits such as BT-H, TT-H, UQT-H, UBT-H, ETT-H, EQT-H. In one example, when the current block is split from a parent block by UBT, the current block is not allowed to be further split. For example, when the current block is split from a parent block by UBT, the current block must be further split, and hence is not allowed to be coded as a single CU.

#### Example 13

(144) In one example, a syntax element such as a flag or an index may be signaled to indicate whether UBT is used.

#### Example 14

(145) In one example, a syntax element such as a flag or an index may be signaled to indicate whether vertical UBT or horizontal UBT is used.

#### Example 15

(146) In one example, a syntax element such as a flag or an index may be signaled to indicate whether Type 0 UBT or Type 1 is used.

#### Example 16

(147) In one example, a syntax element such as a flag or an index may be signaled to indicate the split ratio (such as  $\frac{1}{4}$  or  $\frac{1}{8}$ ) of UBT is used.

(148) FIG. 23 is a block diagram showing an example video processing system **2300** in which various techniques disclosed herein may be implemented. Various implementations may include some or all of the components of the system **2300**. The system **2300** may include input **2302** for receiving video content. The video content may be received in a raw or uncompressed format, e.g., 8 or 10 bit multi-component pixel values, or may be in a compressed or encoded format. The input **2302** may represent a network interface, a peripheral bus interface, or a storage interface. Examples of network interface include wired interfaces such as Ethernet, passive optical network (PON), etc. and wireless interfaces such as Wi-Fi or cellular interfaces.

(149) The system **2300** may include a coding component **2304** that may implement the various coding or encoding methods described in the present document. The coding component **2304** may reduce the average bitrate of video from the input **2302** to the output of the coding component **2304**

to produce a coded representation of the video. The coding techniques are therefore sometimes called video compression or video transcoding techniques. The output of the coding component **2304** may be either stored, or transmitted via a communication connected, as represented by the component **2306**. The stored or communicated bitstream (or coded) representation of the video received at the input **2302** may be used by a component **2308** for generating pixel values or displayable video that is sent to a display interface **2310**. The process of generating user-viewable video from the bitstream representation is sometimes called video decompression. Furthermore, while certain video processing operations are referred to as “coding” operations or tools, it will be appreciated that the coding tools or operations are used at an encoder and corresponding decoding tools or operations that reverse the results of the coding will be performed by a decoder.

(150) Examples of a peripheral bus interface or a display interface may include universal serial bus (USB) or high definition multimedia interface (HDMI) or Displayport, and so on. Examples of storage interfaces include SATA (serial advanced technology attachment), PCI, IDE interface, and the like. The techniques described in the present document may be embodied in various electronic devices such as mobile phones, laptops, smartphones or other devices that are capable of performing digital data processing and/or video display.

(151) FIG. **24** is a block diagram of an example video processing apparatus **2400**. The apparatus **2400** may be used to implement one or more of the methods described herein. The apparatus **2400** may be embodied in a smartphone, tablet, computer, Internet of Things (IoT) receiver, and so on. The apparatus **2400** may include one or more processors **2402**, one or more memories **2404** and video processing circuitry **2406**. The processor(s) **2402** may be configured to implement one or more methods described in the present document. The memory (memories) **2404** may be used for storing data and code used for implementing the methods and techniques described herein. The video processing circuitry **2406** may be used to implement, in hardware circuitry, some techniques described in the present document. In some embodiments, the video processing circuitry **2406** may be at least partly included in the processor **2402**, e.g., a graphics co-processor.

(152) FIG. **25** is a flowchart for an example method **2500** of video processing. The method **2500** includes determining an a UBT partition of a parent block to create two sub-blocks with different dimensions at step **2502**. A UBT partition is a partition that results in at least one sub-block that is dyadic. A dyadic block/sub-block is a block that includes at least one side with a dimension that cannot be expressed as a power of two, which can also be referred to as a dyadic side. Accordingly, at least one of the sub-blocks created by the UBT partition comprises a side that is non-dyadic. In one example, one of the sub-blocks is non-dyadic and one of the sub-blocks is dyadic.

(153) In an example, the UBT partition can be a vertical UBT (UBT-V) partition, which is a UBT partition with a vertical split that creates a first sub-block with a lower index (e.g., zero) and a second sub-block with a higher index (e.g., one). The UBT-V partition is a UBT-V type 0 partition when the first sub-block is smaller than the second sub-block. The UBT-V partition is a UBT-V type 1 partition when the first sub-block is larger than the second sub-block. In another example, the UBT partition can be a horizontal UBT (UBT-H) partition, which is a UBT partition with a horizontal split that creates a first sub-block with a lower index (e.g., zero) and a second sub-block with a higher index (e.g., one). The UBT-H partition is a UBT-H type 0 partition when the first sub-block is smaller than the second sub-block. The UBT-H partition is a UBT-H type 1 partition when the first sub-block is larger than the second sub-block.

(154) At optional step **2504**, the method **2500** can determine to partition a non-dyadic block into sub-blocks. In some examples, the non-dyadic block may be the sub-block created by the UBT partition. In other examples, the non-dyadic block may be a separate block that is not partitioned by UBT partition.

(155) In an example, a width (W) of the non-dyadic block is a non-dyadic number and a height (H) of the non-dyadic block is a dyadic number. In such a case, the partition of the non-dyadic block can be a horizontal partition, such as a horizontal-BT, a horizontal-TT, a horizontal-UQT, a

horizontal-ETT, a horizontal-UBT, etc. In another example, the partition of the non-dyadic block can be a vertical partition. The vertical partition of the non-dyadic block results in a proximate sub-block with a width ( $W1$ ) and a distal sub-block with a width ( $W2$ ).  $W1$  and  $W2$  can be dyadic, non-dyadic, or combinations thereof. Depending on the example,  $W1=W2=W/2$ ,  $W1=\sup.1<< \lfloor \log.\sup.2.\sup.W \rfloor$  and  $W2=W-W1$ ,  $W2=\sup.1<< \lfloor \log.\sup.2.\sup.W \rfloor$  and  $W1=W-W2$ ,  $W1=\sup.1<< \lceil \log.\sup.2.\sup.W \rceil$  and  $W2=W-W1$ ,  $W2=\sup.1<< \lceil \log.\sup.2.\sup.W \rceil$  and  $W1=W-W2$ ,  $W1=1<<N$  and  $W2=W-W1$  and  $N<\sup.1<< \lfloor \log.\sup.2.\sup.W \rfloor$  where  $N$  is a number,  $W2=1<<N$  and  $W1=W-W2$  and  $N<\sup.1<< \lfloor \log.\sup.2.\sup.W \rfloor$  where  $N$  is a number, or combinations thereof. It should be noted that  $<<$  is a left bit shift operation,  $\log$  is a logarithmic operation,  $\lfloor \log.\sup.2.\sup.W \rfloor$  is a logarithmic operation with a floor function, and  $\lceil \log.\sup.2.\sup.W \rceil$  is a logarithmic operation with a ceiling function.

(156) In an example, a  $W$  of the non-dyadic block is a dyadic number and a  $H$  of the non-dyadic block is a non-dyadic number. In such a case, the partition of the non-dyadic block can be a vertical partition, such as a vertical-BT, a vertical-TT, a vertical-UQT, a vertical-ETT, a vertical-UBT, etc. In another example, the partition of the non-dyadic block can be a horizontal partition. The horizontal partition of the non-dyadic block results in a proximate sub-block with a width ( $W1$ ) and a distal sub-block with a width ( $W2$ ).  $W1$  and  $W2$  can be dyadic, non-dyadic, or combinations thereof. Depending on the example,  $H1=H2=H/2$ ,  $H1=\sup.1<< \lfloor \log.\sup.2.\sup.H \rfloor$  and  $H2=H-H1$ ,  $H2=\sup.1<< \lfloor \log.\sup.2.\sup.H \rfloor$  and  $H1=H-H2$ ,  $H1=\sup.1<< \lceil \log.\sup.2.\sup.H \rceil$  and  $H2=H-H1$ ,  $H2=\sup.1<< \lceil \log.\sup.2.\sup.H \rceil$  and  $H1=H-H2$ ,  $H1=1<<N$  and  $H2=H-H1$  and  $N<\sup.1<< \lfloor \log.\sup.2.\sup.H \rfloor$  where  $N$  is a number,  $H2=1<<N$  and  $H1=H-H2$  and  $N<\sup.1<< \lfloor \log.\sup.2.\sup.H \rfloor$  where  $N$  is a number, or combinations thereof.

(157) In an example, a  $W$  of the non-dyadic block is a non-dyadic number and a  $H$  of the non-dyadic block is a non-dyadic number. In an example, the partition of the non-dyadic block can be a vertical partition. The vertical partition of the non-dyadic block results in a proximate sub-block with a width ( $W1$ ) and a distal sub-block with a width ( $W2$ ).  $W1$  and  $W2$  can be dyadic, non-dyadic, or combinations thereof. Depending on the example,  $W1=W2=W/2$ ,  $W1=\sup.1<< \lfloor \log.\sup.2.\sup.W \rfloor$  and  $W2=W-W1$ ,  $W2=\sup.1<< \lfloor \log.\sup.2.\sup.W \rfloor$  and  $W1=W-W2$ ,  $W1=\sup.1<< \lceil \log.\sup.2.\sup.W \rceil$  and  $W2=W-W1$ ,  $W2=\sup.1<< \lceil \log.\sup.2.\sup.W \rceil$  and  $W1=W-W2$ ,  $W1=1<<N$  and  $W2=W-W1$  and  $N<\sup.1<< \lfloor \log.\sup.2.\sup.W \rfloor$  where  $N$  is a number,  $W2=1<<N$  and  $W1=W-W2$  and  $N<\sup.1<< \lfloor \log.\sup.2.\sup.W \rfloor$  where  $N$  is a number, or combinations thereof. In another example, the partition of the non-dyadic block can be a horizontal partition. The horizontal partition of the non-dyadic block results in a proximate sub-block with a width ( $W1$ ) and a distal sub-block with a width ( $W2$ ).  $W1$  and  $W2$  can be dyadic, non-dyadic, or combinations thereof. Depending on the example,  $H1=H2=H/2$ ,  $H1=\sup.1<< \lfloor \log.\sup.2.\sup.H \rfloor$  and  $H2=H-H1$ ,  $H2=\sup.1<< \lfloor \log.\sup.2.\sup.H \rfloor$  and  $H1=H-H2$ ,  $H1=\sup.1<< \lceil \log.\sup.2.\sup.H \rceil$  and  $H2=H-H1$ ,  $H2=\sup.1<< \lceil \log.\sup.2.\sup.H \rceil$  and  $H1=H-H2$ ,  $H1=1<<N$  and  $H2=H-H1$  and  $N<\sup.1<< \lfloor \log.\sup.2.\sup.H \rfloor$  where  $N$  is a number,  $H2=1<<N$  and  $H1=H-H2$  and  $N<\sup.1<< \lfloor \log.\sup.2.\sup.H \rfloor$  where  $N$  is a number, or combinations thereof.

(158) At step **2506**, a conversion is performed between a visual media data and a bitstream based on the sub-blocks. Such a conversion can be performed by encoding at an encoder and/or decoding at a decoder. Such a conversion can be performed according to various rules. For example, a rule may prevent partitioning a sub-block resulting from a UBT partition. In such a case, the conversion process may not include an encoding combination to express partitioning a sub-block resulting from a UBT partition.

(159) Further, such rules may apply different block split constraints to non-dyadic blocks and dyadic blocks. For example, when applying a partition to a dyadic block, a partition tree structure may add a value of one to a bit-depth for the tree, for example by applying  $D+\delta$  where  $\delta$  is zero and  $D$  is the number of splits that occur to partition the largest coding unit into the sub-block of the current split. In contrast, the rule may apply  $D+\delta$  where  $\delta$  may be set to a positive

non-zero integer when applying a split to a non-dyadic block. For example, partitioning a non-dyadic block can add a bit depth greater than one when the non-dyadic block has a side with a measurement that is less than half a measurement of a corresponding side of a parent block, such as three (two plus one) when the sub-block side is  $\frac{1}{4}$  the size of the parent block side size, four (three plus one) when the sub-block side is  $\frac{1}{8}$  the size of the parent block side size, etc. Further, partitioning a non-dyadic block can add a bit depth less than or equal to one when the non-dyadic block has a side with a measurement that is greater than half a measurement of a corresponding side of a parent block, such as one (one plus zero) or two (one plus one) when the sub-block side is  $\frac{3}{4}$  the size of the parent block side size.

(160) Further, in some examples a rule may prevent vertical or horizontal partitioning of any sub-block resulting from UBT-V. In an example, a rule may prevent vertical or horizontal partitioning of any sub-block resulting from UBT-H. In an example, a rule prevents all partitioning of any sub-block resulting from UBT. In some examples, a rule requires further partitioning of any sub-block resulting from UBT, and accordingly prevents the results from UBT becoming coded as a single CU. In an example, the conversion may also involve encoding the UBT split into the bitstream. For example, the bitstream may comprise a syntax element that signals UBT, UBT-V, UBT-H, UBT type 0, UBT type 1, UBT split ratio, or combinations thereof.

(161) It should be noted that the method **2500** can be implemented in an apparatus for processing video data comprising a processor and a non-transitory memory with instructions thereon, such as video encoder **2700**, video decoder **2800**, and/or encoder **2900**. In such a case, the instructions upon execution by the processor, cause the processor to perform the method **2500**. Further, the method **2500** can be performed by a non-transitory computer readable medium comprising a computer program product for use by a video coding device. The computer program product comprises computer executable instructions stored on the non-transitory computer readable medium such that when executed by a processor cause the video coding device to perform the method **2500**.

(162) FIG. **26** is a block diagram that illustrates an example video coding system **2600** that may utilize the techniques of this disclosure. As shown in FIG. **26**, video coding system **2600** may include a source device **2610** and a destination device **2620**. Source device **2610** generates encoded video data which may be referred to as a video encoding device. Destination device **2620** may decode the encoded video data generated by source device **2610** which may be referred to as a video decoding device.

(163) Source device **2610** may include a video source **2612**, a video encoder **2614**, and an input/output (I/O) interface **2616**. Video source **2612** may include a source such as a video capture device, an interface to receive video data from a video content provider, and/or a computer graphics system for generating video data, or a combination of such sources. The video data may comprise one or more pictures. Video encoder **2614** encodes the video data from video source **2612** to generate a bitstream. The bitstream may include a sequence of bits that form a coded representation of the video data. The bitstream may include coded pictures and associated data. The coded picture is a coded representation of a picture. The associated data may include sequence parameter sets, picture parameter sets, and other syntax structures. I/O interface **2616** may include a modulator/demodulator (modem) and/or a transmitter. The encoded video data may be transmitted directly to destination device **2620** via I/O interface **2616** through network **2630**. The encoded video data may also be stored onto a storage medium/server **2640** for access by destination device **2620**.

(164) Destination device **2620** may include an I/O interface **2626**, a video decoder **2624**, and a display device **2622**. I/O interface **2626** may include a receiver and/or a modem. I/O interface **2626** may acquire encoded video data from the source device **2610** or the storage medium/server **2640**. Video decoder **2624** may decode the encoded video data. Display device **2622** may display the decoded video data to a user. Display device **2622** may be integrated with the destination device

**2620**, or may be external to destination device **2620**, which can be configured to interface with an external display device.

(165) Video encoder **2614** and video decoder **2624** may operate according to a video compression standard, such as the High Efficiency Video Coding (HEVC) standard, Versatile Video Coding (VVM) standard and other current and/or further standards.

(166) FIG. **27** is a block diagram illustrating an example of video encoder **2700**, which may be video encoder **2614** in the system **2600** illustrated in FIG. **26**. Video encoder **2700** may be configured to perform any or all of the techniques of this disclosure. In the example of FIG. **27**, video encoder **2700** includes a plurality of functional components. The techniques described in this disclosure may be shared among the various components of video encoder **2700**. In some examples, a processor may be configured to perform any or all of the techniques described in this disclosure.

(167) The functional components of video encoder **2700** may include a partition unit **2701**, a prediction unit **2702** which may include a mode selection unit **2703**, a motion estimation unit **2704**, a motion compensation unit **2705**, an intra prediction unit **2706**, a residual generation unit **2707**, a transform processing unit **2708**, a quantization unit **2709**, an inverse quantization unit **2710**, an inverse transform unit **2711**, a reconstruction unit **2712**, a buffer **2713**, and an entropy encoding unit **2714**.

(168) In other examples, video encoder **2700** may include more, fewer, or different functional components. In an example, prediction unit **2702** may include an intra block copy (IBC) unit. The IBC unit may perform prediction in an MC mode in which at least one reference picture is a picture where the current video block is located.

(169) Furthermore, some components, such as motion estimation unit **2704** and motion compensation unit **2705** may be highly integrated, but are represented in the example of FIG. **27** separately for purposes of explanation.

(170) Partition unit **2701** may partition a picture into one or more video blocks. Video encoder **2700** and video decoder **2800** may support various video block sizes.

(171) Mode selection unit **2703** may select one of the coding modes, intra or inter, e.g., based on error results, and provide the resulting intra or inter coded block to a residual generation unit **2707** to generate residual block data and to a reconstruction unit **2712** to reconstruct the encoded block for use as a reference picture. In some examples, mode selection unit **2703** may select a combination of intra and inter prediction (CIIP) mode in which the prediction is based on an inter prediction signal and an intra prediction signal. Mode selection unit **2703** may also select a resolution for a motion vector (e.g., a sub-pixel or integer pixel precision) for the block in the case of inter prediction.

(172) To perform inter prediction on a current video block, motion estimation unit **2704** may generate motion information for the current video block by comparing one or more reference frames from buffer **2713** to the current video block. Motion compensation unit **2705** may determine a predicted video block for the current video block based on the motion information and decoded samples of pictures from buffer **2713** other than the picture associated with the current video block.

(173) Motion estimation unit **2704** and motion compensation unit **2705** may perform different operations for a current video block, for example, depending on whether the current video block is in an I slice, a P slice, or a B slice.

(174) In some examples, motion estimation unit **2704** may perform uni-directional prediction for the current video block, and motion estimation unit **2704** may search reference pictures of list 0 or list 1 for a reference video block for the current video block. Motion estimation unit **2704** may then generate a reference index that indicates the reference picture in list 0 or list 1 that contains the reference video block and a motion vector that indicates a spatial displacement between the current video block and the reference video block. Motion estimation unit **2704** may output the reference index, a prediction direction indicator, and the motion vector as the motion information of the

current video block. Motion compensation unit **2705** may generate the predicted video block of the current block based on the reference video block indicated by the motion information of the current video block.

(175) In other examples, motion estimation unit **2704** may perform bi-directional prediction for the current video block, motion estimation unit **2704** may search the reference pictures in list 0 for a reference video block for the current video block and may also search the reference pictures in list 1 for another reference video block for the current video block. Motion estimation unit **2704** may then generate reference indexes that indicate the reference pictures in list 0 and list 1 containing the reference video blocks and motion vectors that indicate spatial displacements between the reference video blocks and the current video block. Motion estimation unit **2704** may output the reference indexes and the motion vectors of the current video block as the motion information of the current video block. Motion compensation unit **2705** may generate the predicted video block of the current video block based on the reference video blocks indicated by the motion information of the current video block.

(176) In some examples, motion estimation unit **2704** may output a full set of motion information for decoding processing of a decoder. In some examples, motion estimation unit **2704** may not output a full set of motion information for the current video. Rather, motion estimation unit **2704** may signal the motion information of the current video block with reference to the motion information of another video block. For example, motion estimation unit **2704** may determine that the motion information of the current video block is sufficiently similar to the motion information of a neighboring video block.

(177) In one example, motion estimation unit **2704** may indicate, in a syntax structure associated with the current video block, a value that indicates to the video decoder **2800** that the current video block has the same motion information as another video block.

(178) In another example, motion estimation unit **2704** may identify, in a syntax structure associated with the current video block, another video block and a motion vector difference (MVD). The motion vector difference indicates a difference between the motion vector of the current video block and the motion vector of the indicated video block. The video decoder **2800** may use the motion vector of the indicated video block and the motion vector difference to determine the motion vector of the current video block.

(179) As discussed above, video encoder **2700** may predictively signal the motion vector. Two examples of predictive signaling techniques that may be implemented by video encoder **2700** include advanced motion vector prediction (AMVP) and merge mode signaling.

(180) Intra prediction unit **2706** may perform intra prediction on the current video block. When intra prediction unit **2706** performs intra prediction on the current video block, intra prediction unit **2706** may generate prediction data for the current video block based on decoded samples of other video blocks in the same picture. The prediction data for the current video block may include a predicted video block and various syntax elements.

(181) Residual generation unit **2707** may generate residual data for the current video block by subtracting the predicted video block(s) of the current video block from the current video block. The residual data of the current video block may include residual video blocks that correspond to different sample components of the samples in the current video block.

(182) In other examples, there may be no residual data for the current video block for the current video block, for example in a skip mode, and residual generation unit **2707** may not perform the subtracting operation.

(183) Transform processing unit **2708** may generate one or more transform coefficient video blocks for the current video block by applying one or more transforms to a residual video block associated with the current video block.

(184) After transform processing unit **2708** generates a transform coefficient video block associated with the current video block, quantization unit **2709** may quantize the transform coefficient video

block associated with the current video block based on one or more quantization parameter (QP) values associated with the current video block.

(185) Inverse quantization unit **2710** and inverse transform unit **2711** may apply inverse quantization and inverse transforms to the transform coefficient video block, respectively, to reconstruct a residual video block from the transform coefficient video block. Reconstruction unit **2712** may add the reconstructed residual video block to corresponding samples from one or more predicted video blocks generated by the prediction unit **2702** to produce a reconstructed video block associated with the current block for storage in the buffer **2713**.

(186) After reconstruction unit **2712** reconstructs the video block, the loop filtering operation may be performed to reduce video blocking artifacts in the video block.

(187) Entropy encoding unit **2714** may receive data from other functional components of the video encoder **2700**. When entropy encoding unit **2714** receives the data, entropy encoding unit **2714** may perform one or more entropy encoding operations to generate entropy encoded data and output a bitstream that includes the entropy encoded data.

(188) FIG. **28** is a block diagram illustrating an example of video decoder **2800** which may be video decoder **2624** in the system **2600** illustrated in FIG. **26**.

(189) The video decoder **2800** may be configured to perform any or all of the techniques of this disclosure. In the example of FIG. **28**, the video decoder **2800** includes a plurality of functional components. The techniques described in this disclosure may be shared among the various components of the video decoder **2800**. In some examples, a processor may be configured to perform any or all of the techniques described in this disclosure.

(190) In the example of FIG. **28**, video decoder **2800** includes an entropy decoding unit **2801**, a motion compensation unit **2802**, an intra prediction unit **2803**, an inverse quantization unit **2804**, an inverse transformation unit **2805**, and a reconstruction unit **2806** and a buffer **2807**. Video decoder **2800** may, in some examples, perform a decoding pass generally reciprocal to the encoding pass described with respect to video encoder **2700** (FIG. **27**).

(191) Entropy decoding unit **2801** may retrieve an encoded bitstream. The encoded bitstream may include entropy coded video data (e.g., encoded blocks of video data). Entropy decoding unit **2801** may decode the entropy coded video data, and from the entropy decoded video data, motion compensation unit **2802** may determine motion information including motion vectors, motion vector precision, reference picture list indexes, and other motion information. Motion compensation unit **2802** may, for example, determine such information by performing the AMVP and merge mode.

(192) Motion compensation unit **2802** may produce motion compensated blocks, possibly performing interpolation based on interpolation filters. Identifiers for interpolation filters to be used with sub-pixel precision may be included in the syntax elements.

(193) Motion compensation unit **2802** may use interpolation filters as used by video encoder **2700** during encoding of the video block to calculate interpolated values for sub-integer pixels of a reference block. Motion compensation unit **2802** may determine the interpolation filters used by video encoder **2700** according to received syntax information and use the interpolation filters to produce predictive blocks.

(194) Motion compensation unit **2802** may use some of the syntax information to determine sizes of blocks used to encode frame(s) and/or slice(s) of the encoded video sequence, partition information that describes how each macroblock of a picture of the encoded video sequence is partitioned, modes indicating how each partition is encoded, one or more reference frames (and reference frame lists) for each inter coded block, and other information to decode the encoded video sequence.

(195) Intra prediction unit **2803** may use intra prediction modes for example received in the bitstream to form a prediction block from spatially adjacent blocks. Inverse quantization unit **2804** inverse quantizes, i.e., de-quantizes, the quantized video block coefficients provided in the

bitstream and decoded by entropy decoding unit **2801**. Inverse transform unit **2805** applies an inverse transform.

(196) Reconstruction unit **2806** may sum the residual blocks with the corresponding prediction blocks generated by motion compensation unit **2802** or intra prediction unit **2803** to form decoded blocks. If desired, a deblocking filter may also be applied to filter the decoded blocks in order to remove blockiness artifacts. The decoded video blocks are then stored in buffer **2807**, which provides reference blocks for subsequent motion compensation/intra prediction and also produces decoded video for presentation on a display device.

(197) FIG. **29** is a schematic diagram of an example encoder **2900**. The encoder **2900** is suitable for implementing the techniques of VVC. The encoder **2900** includes three in-loop filters, namely a deblocking filter (DF) **2902**, a sample adaptive offset (SAO) **2904**, and an adaptive loop filter (ALF) **2906**. Unlike the DF **2902**, which uses predefined filters, the SAO **2904** and the ALF **2906** utilize the original samples of the current picture to reduce the mean square errors between the original samples and the reconstructed samples by adding an offset and by applying a finite impulse response (FIR) filter, respectively, with coded side information signaling the offsets and filter coefficients. The ALF **2906** is located at the last processing stage of each picture and can be regarded as a tool trying to catch and fix artifacts created by the previous stages.

(198) The encoder **2900** further includes an intra prediction component **2908** and a motion estimation/compensation (ME/MC) component **2910** configured to receive input video. The intra prediction component **2908** is configured to perform intra prediction, while the ME/MC component **2910** is configured to utilize reference pictures obtained from a reference picture buffer **2912** to perform inter prediction. Residual blocks from inter prediction or intra prediction are fed into a transform (T) component **2914** and a quantization (Q) component **2916** to generate quantized residual transform coefficients, which are fed into an entropy coding component **2918**. The entropy coding component **2918** entropy codes the prediction results and the quantized transform coefficients and transmits the same toward a video decoder (not shown). Quantization components output from the quantization component **2916** may be fed into an inverse quantization (IQ) components **2920**, an inverse transform component **2922**, and a reconstruction (REC) component **2924**. The REC component **2924** is able to output images to the DF **2902**, the SAO **2904**, and the ALF **2906** for filtering prior to those images being stored in the reference picture buffer **2912**.

(199) A listing of solutions preferred by some examples is provided next.

(200) The following solutions show examples of techniques discussed herein.

(201) 1. A method of video processing (e.g., method **2500** depicted in FIG. **25**), comprising performing a conversion between a video block of a video and a coded representation of the video; wherein the video block has a dimension of  $W \times H$  samples, wherein, for the conversion, the video block is partitioned using an unsymmetric binary tree (UBT) split such that the video block is divided into two sub-blocks of  $W1 \times H1$  and  $W2 \times H2$ , having different dimensions.

(202) 2. The method of solution 1, wherein at least one of  $W1$ ,  $H1$ ,  $W2$ ,  $H2$  is non-dyadic.

(203) 3. The method of any of any of solutions 1-2, wherein one of the two sub-blocks is dyadic and the other of the two sub-blocks is non-dyadic.

(204) 4. The method of solution 1, wherein the two sub-blocks are dividing using a vertical UBT in which  $W1 = a \times W$ ,  $W2 = (1-a) \times W$ ,  $H1 = H2 = H$ , where  $a$  is a fraction.

(205) 5. The method of solution 1, wherein the two sub-blocks are dividing using a horizontal UBT in which  $H1 = a \times H$ ,  $H2 = (1-a) \times H$ ,  $W1 = W2 = W$ , where  $a$  is a fraction.

(206) 6. A method of video processing, comprising: performing a conversion between a video block of a video and a coded representation of the video; wherein the video block is a non-dyadic block having dimensions of  $W \times H$  samples, wherein, for the conversion, the video block is split into sub-blocks according to a rule.

(207) 7. The method of solution 6, wherein the rule specifies that, responsive to  $W$  being non-dyadic and  $H$  being dyadic, a horizontal split method is used for splitting the video block into sub-



blocks.

(208) 8. The method of solution 6, wherein the rule specifies that, responsive to W being non-dyadic and H being dyadic, a vertical split method is used for splitting the video block into sub-blocks.

(209) 9. The method of solution 6, wherein the rule specifies that, responsive to W being dyadic and H being non-dyadic, a horizontal split method is used for splitting the video block into sub-blocks.

(210) 10. The method of solution 6, wherein the rule specifies that, responsive to W being dyadic and H being non-dyadic, a vertical split method is used for splitting the video block into sub-blocks.

(211) 11. The method of solution 6, wherein the rule specifies that, responsive to W being non-dyadic and H being non-dyadic, a horizontal split method is used for splitting the video block into sub-blocks.

(212) 12. The method of solution 6, wherein the rule specifies that, responsive to W being non-dyadic and H being non-dyadic, a vertical split method is used for splitting the video block into sub-blocks.

(213) 13. A method of video processing, comprising: performing a conversion between a video comprising a video block and a bitstream representation of the video, wherein a rule specifies whether or how the video block is partitioned into sub-blocks according to whether an unsymmetric binary tree (UBT) split is used.

(214) 14. The method of solution 13, wherein the rule specifies that a tree-depth of splitting the video block is dependent on whether the UBT split is used.

(215) 15. The method of solution 13, wherein the rule specifies that, in case that the video block is obtained from a parent block using a vertical UBT split, then the video block is disallowed from a further vertical split.

(216) 16. The method of solution 13, wherein the rule specifies that, in case that the video block is obtained from a parent block using a horizontal UBT split, then the video block is disallowed from a further vertical split.

(217) 17. The method of solution 13, wherein the rule specifies that, in case that the video block is obtained from a parent block using a horizontal UBT split, then the video block is disallowed from a further horizontal split.

(218) 18. The method of solution 13, wherein the rule specifies that, in case that the video block is obtained from a parent block using a vertical UBT split, then the video block is disallowed from a further horizontal split.

(219) The following solutions show example embodiments of techniques discussed in the previous section (e.g., items 4-7).

(220) 19. The method of any of solutions 1-18, wherein the bitstream includes an indication of whether or how the UBT split is used for the conversion.

(221) 20. The method of solution 19, wherein the indication indicates whether a vertical UBT split or a horizontal UBT split is used.

(222) 21. The method of solution 19-20, wherein the indication indicates whether a Type 0 or a Type 1 UBT split is used.

(223) 22. The method of any of solutions 19-21, wherein the indication indicates a split ration used for the UBT split.

(224) 23. The method of any of solutions 1-22, wherein the conversion includes generating the bitstream from the video.

(225) 24. The method of any of solutions 1-22, wherein the conversion includes generating the video from the bitstream.

(226) 25. A method of storing a bitstream on a computer-readable medium, comprising generating a bitstream according to a method recited in any one or more of solutions 1-23, and storing the

bitstream on the computer-readable medium.

(227) 26. A computer-readable medium having a bitstream of a video stored thereon, the bitstream, when processed by a processor of a video decoder, causing the video decoder to generate the video, wherein the bitstream is generated according to a method recited in one or more of solutions 1-23.

(228) 27. A video decoding apparatus comprising a processor configured to implement a method recited in one or more of solutions 1 to 23.

(229) 28. A video encoding apparatus comprising a processor configured to implement a method recited in one or more of solutions 1 to 23.

(230) 29. A computer program product having computer code stored thereon, the code, when executed by a processor, causes the processor to implement a method recited in any of solutions 1 to 23.

(231) 30. A computer readable medium on which a bitstream complying to a bitstream format that is generated according to any of solutions 1 to 23.

(232) 31. A method, an apparatus, a bitstream generated according to a disclosed method or a system described in the present document.

(233) In the solutions described herein, an encoder may conform to the format rule by producing a coded representation according to the format rule. In the solutions described herein, a decoder may use the format rule to parse syntax elements in the coded representation with the knowledge of presence and absence of syntax elements according to the format rule to produce decoded video.

(234) In the present document, the term “video processing” may refer to video encoding, video decoding, video compression or video decompression. For example, video compression algorithms may be applied during conversion from pixel representation of a video to a corresponding bitstream representation or vice versa. The bitstream representation of a current video block may, for example, correspond to bits that are either co-located or spread in different places within the bitstream, as is defined by the syntax. For example, a macroblock may be encoded in terms of transformed and coded error residual values and also using bits in headers and other fields in the bitstream. Furthermore, during conversion, a decoder may parse a bitstream with the knowledge that some fields may be present, or absent, based on the determination, as is described in the above solutions. Similarly, an encoder may determine that certain syntax fields are or are not to be included and generate the coded representation accordingly by including or excluding the syntax fields from the coded representation.

(235) The disclosed and other solutions, examples, embodiments, modules and the functional operations described in this document can be implemented in digital electronic circuitry, or in computer software, firmware, or hardware, including the structures disclosed in this document and their structural equivalents, or in combinations of one or more of them. The disclosed and other embodiments can be implemented as one or more computer program products, i.e., one or more modules of computer program instructions encoded on a computer readable medium for execution by, or to control the operation of, data processing apparatus. The computer readable medium can be a machine-readable storage device, a machine-readable storage substrate, a memory device, a composition of matter effecting a machine-readable propagated signal, or a combination of one or more them. The term “data processing apparatus” encompasses all apparatus, devices, and machines for processing data, including by way of example a programmable processor, a computer, or multiple processors or computers. The apparatus can include, in addition to hardware, code that creates an execution environment for the computer program in question, e.g., code that constitutes processor firmware, a protocol stack, a database management system, an operating system, or a combination of one or more of them. A propagated signal is an artificially generated signal, e.g., a machine-generated electrical, optical, or electromagnetic signal, that is generated to encode information for transmission to suitable receiver apparatus.

(236) A computer program (also known as a program, software, software application, script, or code) can be written in any form of programming language, including compiled or interpreted

languages, and it can be deployed in any form, including as a stand-alone program or as a module, component, subroutine, or other unit suitable for use in a computing environment. A computer program does not necessarily correspond to a file in a file system. A program can be stored in a portion of a file that holds other programs or data (e.g., one or more scripts stored in a markup language document), in a single file dedicated to the program in question, or in multiple coordinated files (e.g., files that store one or more modules, sub programs, or portions of code). A computer program can be deployed to be executed on one computer or on multiple computers that are located at one site or distributed across multiple sites and interconnected by a communication network.

(237) The processes and logic flows described in this document can be performed by one or more programmable processors executing one or more computer programs to perform functions by operating on input data and generating output. The processes and logic flows can also be performed by, and apparatus can also be implemented as, special purpose logic circuitry, e.g., an FPGA (field programmable gate array) or an ASIC (application specific integrated circuit).

(238) Processors suitable for the execution of a computer program include, by way of example, both general and special purpose microprocessors, and any one or more processors of any kind of digital computer. Generally, a processor will receive instructions and data from a read only memory or a random-access memory or both. The essential elements of a computer are a processor for performing instructions and one or more memory devices for storing instructions and data.

Generally, a computer will also include, or be operatively coupled to receive data from or transfer data to, or both, one or more mass storage devices for storing data, e.g., magnetic, magneto optical disks, or optical disks. However, a computer need not have such devices. Computer readable media suitable for storing computer program instructions and data include all forms of non-volatile memory, media and memory devices, including by way of example semiconductor memory devices, e.g., erasable programmable read-only memory (EPROM), electrically erasable programmable read-only memory (EEPROM), and flash memory devices; magnetic disks, e.g., internal hard disks or removable disks; magneto optical disks; and compact disc read-only memory (CD ROM) and Digital versatile disc-read only memory (DVD-ROM) disks. The processor and the memory can be supplemented by, or incorporated in, special purpose logic circuitry.

(239) While this patent document contains many specifics, these should not be construed as limitations on the scope of any subject matter or of what may be claimed, but rather as descriptions of features that may be specific to particular embodiments of particular techniques. Certain features that are described in this patent document in the context of separate embodiments can also be implemented in combination in a single embodiment. Conversely, various features that are described in the context of a single embodiment can also be implemented in multiple embodiments separately or in any suitable subcombination. Moreover, although features may be described above as acting in certain combinations and even initially claimed as such, one or more features from a claimed combination can in some cases be excised from the combination, and the claimed combination may be directed to a subcombination or variation of a subcombination.

(240) Similarly, while operations are depicted in the drawings in a particular order, this should not be understood as requiring that such operations be performed in the particular order shown or in sequential order, or that all illustrated operations be performed, to achieve desirable results. Moreover, the separation of various system components in the embodiments described in this patent document should not be understood as requiring such separation in all embodiments.

(241) Only a few implementations and examples are described and other implementations, enhancements and variations can be made based on what is described and illustrated in this patent document.

(242) A first component is directly coupled to a second component when there are no intervening components, except for a line, a trace, or another medium between the first component and the second component. The first component is indirectly coupled to the second component when there

are intervening components other than a line, a trace, or another medium between the first component and the second component. The term “coupled” and its variants include both directly coupled and indirectly coupled. The use of the term “about” means a range including  $\pm 10\%$  of the subsequent number unless otherwise stated.

(243) While several embodiments have been provided in the present disclosure, it should be understood that the disclosed systems and methods might be embodied in many other specific forms without departing from the spirit or scope of the present disclosure. The present examples are to be considered as illustrative and not restrictive, and the intention is not to be limited to the details given herein. For example, the various elements or components may be combined or integrated in another system or certain features may be omitted, or not implemented.

(244) In addition, techniques, systems, subsystems, and methods described and illustrated in the various embodiments as discrete or separate may be combined or integrated with other systems, modules, techniques, or methods without departing from the scope of the present disclosure. Other items shown or discussed as coupled may be directly connected or may be indirectly coupled or communicating through some interface, device, or intermediate component whether electrically, mechanically, or otherwise. Other examples of changes, substitutions, and alterations are ascertainable by one skilled in the art and could be made without departing from the spirit and scope disclosed herein.

## Claims

1. A method for processing video data, comprising: determining, during a conversion between a current video block with a dimension of  $W \times H$  of a video and a bitstream of the video, an unsymmetric binary tree (UBT) partition scheme is applied to the current video block, and the current video block is split into a first sub-block with a first dimension  $W1 \times H1$  and a second sub-block with a second dimension  $W2 \times H2$ , and wherein the first dimension is different from the second dimension, where  $W$  indicates a width of the current video block and  $H$  indicates a height of the current video block,  $W1$  indicates a width of the first sub-block and  $H1$  indicates a height of the first sub-block,  $W2$  indicates a width of the second sub-block and  $H2$  indicates a height of the second sub-block, wherein  $W$ ,  $H$ ,  $W1$ ,  $H1$ ,  $W2$  and  $H2$  are positive integers; and performing the conversion based on the sub-blocks, wherein if the current video block is split from a parent block which is split by UBT-V, then the current video block is prevented from being further split by vertical splits, wherein the vertical splits includes BT-V, TT-V, UQT-V, UBT-V, ETT-V, and EQT-V; wherein if the current video block is split from a parent block which is split by UBT-H, then the current video block is prevented from being further split by vertical splits; wherein if the current video block is split from a parent block which is split by UBT-V, then the current video block is prevented from being further split by horizontal splits, wherein the horizontal splits includes BT-H, TT-H, UQT-H, UBT-H, ETT-H, and EQT-H; wherein if the current video block is split from a parent block which is split by UBT-H, then the current video block is prevented from being further split by horizontal splits; or wherein if the current block is split from a parent block by UBT, it is not allowed to be further split; or alternatively, if the current video block is split from a parent block which is split by UBT, the current video block must be further split.

2. The method of claim 1, wherein at least one of the  $W1$ ,  $H1$ ,  $W2$ ,  $H2$  has a non-dyadic value, wherein the non-dyadic value indicates a value that cannot be represented in a form of a power of 2.

3. The method of claim 1, wherein one of the first sub-block and the second sub-block is a non-dyadic block and the other one of the first sub-block and the second subblock is a dyadic block, wherein both width and height of the dyadic block have dyadic values, and at least one of width and height of the non-dyadic block has a non-dyadic value.

4. The method of claim 1, wherein when  $W1 = a \times W$ ,  $W2 = (1 - a) \times W$ ,  $H1 = H2 = H$ , the UBT partition is

a vertical UBT (UBT-V) partition; wherein the UBT-V partition is a UBT-V type 0 partition when  $a$  is smaller than  $\frac{1}{2}$ , and wherein the UBT-V partition is a UBT-V type 1 partition when  $a$  is larger than  $\frac{1}{2}$ ; wherein a coding order of the two sub-blocks is from left to right; and wherein when  $a$  is smaller than  $\frac{1}{2}$ ,  $a$  is equal to  $\frac{1}{4}$ ,  $\frac{1}{8}$ ,  $\frac{1}{16}$ ,  $\frac{1}{32}$ , or  $\frac{1}{64}$ , and when  $a$  is larger than  $\frac{1}{2}$ ,  $a$  is equal to  $\frac{3}{4}$ ,  $\frac{7}{8}$ ,  $\frac{15}{16}$ ,  $\frac{31}{32}$ , or  $\frac{63}{64}$ .

5. The method of claim 1, wherein when  $H1=a \times H$ ,  $H2=(1-a) \times H$ ,  $W1=W2=W$ , the UBT partition is a horizontal UBT (UBT-H) partition; wherein the UBT-H partition is a UBT-H type 0 partition when  $a$  is smaller than  $\frac{1}{2}$ , and wherein the UBT-H partition is a UBT-H type 1 partition when  $a$  is larger than  $\frac{1}{2}$ ; wherein a coding order of the two sub-blocks is from top to bottom; and wherein when  $a$  is smaller than  $\frac{1}{2}$ ,  $a$  is equal to  $\frac{1}{4}$ ,  $\frac{1}{8}$ ,  $\frac{1}{16}$ ,  $\frac{1}{32}$ , or  $\frac{1}{64}$ , and when  $a$  is larger than  $\frac{1}{2}$ ,  $a$  is equal to  $\frac{3}{4}$ ,  $\frac{7}{8}$ ,  $\frac{15}{16}$ ,  $\frac{31}{32}$ , or  $\frac{63}{64}$ .

6. The method of claim 1, wherein when at least one of  $W$  and  $H$  has a non-dyadic value, the current video block is partitioned into sub-blocks.

7. The method of claim 6, wherein when  $W$  has a non-dyadic value and  $H$  has a dyadic value: the current video block is partitioned with a horizontal partition scheme, wherein the horizontal partition scheme includes at least one of a horizontal-BT, horizontal-TT, horizontal-UQT, horizontal-ETT, or horizontal-UBT partition schemes; or the current video block is partitioned with a vertical partition scheme and  $H1=H2=H$ , wherein  $W1$  and  $W2$  satisfies one of the following conditions: i. at least one of  $W1$  and  $W2$  has a dyadic value; ii. both  $W1$  and  $W2$  have dyadic values; iii. both  $W1$  and  $W2$  have non-dyadic values; iv.  $W1=W2=W/2$ ; v.  $W1=1 \ll \lfloor \log_{\text{sub.2}} W \rfloor$ ,  $W2=W-W1$ ; vi.  $W2=1 \ll \lfloor \log_{\text{sub.2}} W \rfloor$ ,  $W1=W-W2$ ; vii.  $W1=1 \ll \lceil \log_{\text{sub.2}} W \rceil$ ,  $W2=W-W1$ ; viii.  $W2=1 \ll \lceil \log_{\text{sub.2}} W \rceil$ ,  $W1=W-W2$ ; ix.  $W1=1 \ll N$ ,  $W2=W-W1$ , wherein  $N < 1 \ll \lfloor \log_{\text{sub.2}} W \rfloor$ ; x.  $W2=1 \ll N$ ,  $W1=W-W2$ , wherein  $N < 1 \ll \lfloor \log_{\text{sub.2}} W \rfloor$ ; or the vertical partition scheme includes at least one of: vertical TT, ETT, QT, EQT, or UQT partition schemes.

8. The method of claim 6, wherein when  $H$  has a non-dyadic value and  $W$  has a dyadic value: the current video block is partitioned with a vertical partition scheme, wherein the vertical partition scheme includes at least one of a vertical-BT, vertical-TT, vertical-UQT, vertical-ETT, or vertical-UBT partition schemes; or the current video block is partitioned with a horizontal partition scheme and  $W1=W2=W$ , wherein  $H1$  and  $H2$  satisfies one of the following conditions: i. at least one of  $H1$  and  $H2$  has a dyadic value; ii. both  $H1$  and  $H2$  have dyadic values; iii. both  $H1$  and  $H2$  have non-dyadic values; iv.  $H1=H2=H/2$ ; v.  $H1=1 \ll \lfloor \log_{\text{sub.2}} H \rfloor$ ,  $H2=H-H1$ ; vi.  $H2=1 \ll \lfloor \log_{\text{sub.2}} H \rfloor$ ,  $H1=H-H2$ ; vii.  $H1=1 \ll \lceil \log_{\text{sub.2}} H \rceil$ ,  $H2=H-H1$ ; viii.  $H2=1 \ll \lceil \log_{\text{sub.2}} H \rceil$ ,  $H1=H-H2$ ; ix.  $H1=1 \ll N$ ,  $H2=H-H1$ , wherein  $N < 1 \ll \lfloor \log_{\text{sub.2}} H \rfloor$ ; x.  $H2=1 \ll N$ ,  $H1=H-H2$ , wherein  $N < 1 \ll \lfloor \log_{\text{sub.2}} H \rfloor$ ; or the horizontal partition scheme includes at least one of: horizontal TT, ETT, QT, EQT, or UQT partition schemes.

9. The method of claim 6, wherein when both  $W$  and  $H$  have non-dyadic values: the current video block is partitioned with a vertical partition scheme and  $H1=H2=H$ , wherein  $W1$  and  $W2$  satisfies one of the following conditions: i. at least one of  $W1$  and  $W2$  has a dyadic value; ii. both  $W1$  and  $W2$  have dyadic values; iii. both  $W1$  and  $W2$  have non-dyadic values; iv.  $W1=W2=W/2$ ; v.  $W1=1 \ll \lfloor \log_{\text{sub.2}} W \rfloor$ ,  $W2=W-W1$ ; vi.  $W2=1 \ll \lfloor \log_{\text{sub.2}} W \rfloor$ ,  $W1=W-W2$ ; vii.  $W1=1 \ll \lceil \log_{\text{sub.2}} W \rceil$ ,  $W2=W-W1$ ; viii.  $W2=1 \ll \lceil \log_{\text{sub.2}} W \rceil$ ,  $W1=W-W2$ ; ix.  $W1=1 \ll N$ ,  $W2=W-W1$ , wherein  $N < 1 \ll \lfloor \log_{\text{sub.2}} W \rfloor$ ; x.  $W2=1 \ll N$ ,  $W1=W-W2$ , wherein  $N < 1 \ll \lfloor \log_{\text{sub.2}} W \rfloor$ ; or the vertical partition scheme includes at least one of: vertical TT, ETT, QT, EQT, or UQT partition schemes; or the current video block is partitioned with a horizontal partition scheme and  $W1=W2=W$ , wherein  $H1$  and  $H2$  satisfies one of the following conditions: i. at least one of  $H1$  and  $H2$  has a dyadic value; ii. both  $H1$  and  $H2$  have dyadic values; iii. both  $H1$  and  $H2$  have non-dyadic values; iv.  $H1=H2=H/2$ ; v.  $H1=1 \ll \lfloor \log_{\text{sub.2}} H \rfloor$ ,  $H2=H-H1$ ; vi.  $H2=1 \ll \lfloor \log_{\text{sub.2}} H \rfloor$ ,  $H1=H-H2$ ; vii.  $H1=1 \ll \lceil \log_{\text{sub.2}} H \rceil$ ,  $H2=H-H1$ ; viii.  $H2=1 \ll \lceil \log_{\text{sub.2}} H \rceil$ ,  $H1=H-H2$ ; ix.  $H1=1 \ll N$ ,  $H2=H-H1$ , wherein  $N < 1 \ll \lfloor \log_{\text{sub.2}} H \rfloor$ ; xi.  $H2=1 \ll N$ ,  $H1=H-H2$ ,

wherein  $N < 1 < \lceil \log_2 H \rceil$ ; or the horizontal partition scheme includes at least one of: horizontal TT, ETT, QT, EQT, or UQT partition schemes.

10. The method of claim 1, wherein when at least one of W and H has a non-dyadic value, the current video block is disallowed to be partitioned into sub-blocks; and wherein an indication of UBT for sub-blocks is not present in the bitstream.

11. The method of claim 1, wherein a first rule is applied to the current video block when the current video block is a non-dyadic block, and a second rule is applied to the current video block when the current video block is a dyadic block, wherein the first rule is different from the second rule.

12. The method of claim 1, wherein a first tree-depth of the current video block when it is partitioned by UBT is different from a second tree-depth of the current video block when it is partitioned by BT, and wherein the first tree-depth is added by a delta, where delta is an integer; wherein a multiple-tree depth (MT-depth) or a binary-tree depth (BT-depth) of a sub-block split from the current video block is derived as  $D + \text{delta}$ , wherein D is the MT-depth of the current block to be split by UBT; wherein  $\text{delta} > 1$  if the dimension of the sub-block is smaller than half of the dimension of the current video block; wherein  $\text{delta} \leq 1$  if the dimension of the sub-block is larger than half of the dimension of the current block; wherein  $\text{delta} = 2$  if the dimension of the sub-block is  $\frac{1}{4}$  of the dimension of the current video block; wherein  $\text{delta} = 3$  if the dimension of the sub-block is  $\frac{1}{8}$  of the dimension of the current video block; and wherein  $\text{delta} = 1$  or 0 if the the dimension of sub-block is  $\frac{3}{4}$  of the dimension of the current video block.

13. The method of claim 1, wherein the bitstream comprises at least one of: a first syntax element indicating whether UBT is enabled; a second syntax element indicating whether UBT-V or UBT-H is enabled; a third syntax element indicating whether UBT type 0 or UBT type 1 is enabled; and a fourth syntax element indicating a split ratio of UBT.

14. The method of claim 13, wherein each of the first, second, third, fourth syntax elements is a flag or an index.

15. The method of claim 1, wherein the conversion includes encoding the video into the bitstream.

16. The method of claim 1, wherein the conversion includes decoding the video from the bitstream.

17. An apparatus for processing video data comprising a processor and a non-transitory memory with instructions thereon, wherein the instructions upon execution by the processor, cause the processor to: determine, during a conversion between a current video block with a dimension of  $W \times H$  of a video and a bitstream of the video, an unsymmetric binary tree (UBT) partition scheme is applied to the current video block, and the current video block is split into a first sub-block with a first dimension  $W1 \times H1$  and a second sub-block with a second dimension  $W2 \times H2$ , and wherein the first dimension is different from the second dimension, where W indicates a width of the current video block and H indicates a height of the current video block, W1 indicates a width of the first sub-block and H1 indicates a height of the first sub-block, W2 indicates a width of the second sub-block and H2 indicates a height of the second sub-block, wherein W, H, W1, H1, W2 and H2 are positive integers; and perform the conversion based on the sub-blocks, wherein if the current video block is split from a parent block which is split by UBT-V, then the current video block is prevented from being further split by vertical splits, wherein the vertical splits includes BT-V, TT-V, UQT-V, UBT-V, ETT-V, and EQT-V; wherein if the current video block is split from a parent block which is split by UBT-H, then the current video block is prevented from being further split by vertical splits; wherein if the current video block is split from a parent block which is split by UBT-V, then the current video block is prevented from being further split by horizontal splits, wherein the horizontal splits includes BT-H, TT-H, UQT-H, UBT-H, ETT-H, and EQT-H; wherein if the current video block is split from a parent block which is split by UBT-H, then the current video block is prevented from being further split by horizontal splits; or wherein if the current block is split from a parent block by UBT, it is not allowed to be further split; or alternatively, if the current video block is split from a parent block which is split by UBT, the current video block must

be further split.

18. The apparatus of claim 17, wherein at least one of the W1, H1, W2, H2 has a non-dyadic value, wherein the non-dyadic value indicates a value that cannot be represented in a form of a power of 2.

19. A non-transitory computer-readable storage medium storing instructions that cause a processor to: determine, during a conversion between a current video block with a dimension of  $W \times H$  of a video and a bitstream of the video, an unsymmetric binary tree (UBT) partition scheme is applied to the current video block, and the current video block is split into a first sub-block with a first dimension  $W1 \times H1$  and a second sub-block with a second dimension  $W2 \times H2$ , and wherein the first dimension is different from the second dimension, where W indicates a width of the current video block and H indicates a height of the current video block, W1 indicates a width of the first sub-block and H1 indicates a height of the first sub-block, W2 indicates a width of the second sub-block and H2 indicates a height of the second sub-block, wherein W, H, W1, H1, W2 and H2 are positive integers; and perform the conversion based on the sub-blocks, wherein if the current video block is split from a parent block which is split by UBT-V, then the current video block is prevented from being further split by vertical splits, wherein the vertical splits includes BT-V, TT-V, UQT-V, UBT-V, ETT-V, and EQT-V; wherein if the current video block is split from a parent block which is split by UBT-H, then the current video block is prevented from being further split by vertical splits; wherein if the current video block is split from a parent block which is split by UBT-V, then the current video block is prevented from being further split by horizontal splits, wherein the horizontal splits includes BT-H, TT-H, UQT-H, UBT-H, ETT-H, and EQT-H; wherein if the current video block is split from a parent block which is split by UBT-H, then the current video block is prevented from being further split by horizontal splits; or wherein if the current block is split from a parent block by UBT, it is not allowed to be further split; or alternatively, if the current video block is split from a parent block which is split by UBT, the current video block must be further split.

20. A method for storing a bitstream of a video, comprising: determining, for a current video block with a dimension of  $W \times H$  of a video, an unsymmetric binary tree (UBT) partition scheme is applied to the current video block, and the current video block is split into a first sub-block with a first dimension  $W1 \times H1$  and a second sub-block with a second dimension  $W2 \times H2$ , and wherein the first dimension is different from the second dimension, where W indicates a width of the current video block and H indicates a height of the current video block, W1 indicates a width of the first sub-block and H1 indicates a height of the first sub-block, W2 indicates a width of the second sub-block and H2 indicates a height of the second sub-block, wherein W, H, W1, H1, W2 and H2 are positive integers; generating the bitstream based on the sub-blocks; and storing the bitstream in a non-transitory computer-readable recording medium, wherein if the current video block is split from a parent block which is split by UBT-V, then the current video block is prevented from being further split by vertical splits, wherein the vertical splits includes BT-V, TT-V, UQT-V, UBT-V, ETT-V, and EQT-V; wherein if the current video block is split from a parent block which is split by UBT-H, then the current video block is prevented from being further split by vertical splits; wherein if the current video block is split from a parent block which is split by UBT-V, then the current video block is prevented from being further split by horizontal splits, wherein the horizontal splits includes BT-H, TT-H, UQT-H, UBT-H, ETT-H, and EQT-H; wherein if the current video block is split from a parent block which is split by UBT-H, then the current video block is prevented from being further split by horizontal splits; or wherein if the current block is split from a parent block by UBT, it is not allowed to be further split; or alternatively, if the current video block is split from a parent block which is split by UBT, the current video block must be further split.

---